Table of Contents

1. Informative Part . . . 2
   1.1. Team . . . 2
   1.2. Current Situation, Needs, and Ideas . . . 4
   1.3. Scope, Span, and Synopsis . . . 8
   1.4. Foundational Activities for System Development . . . 10
   1.5. Derived Goals . . . 11
2. Descriptive Part . . . 15
   2.1. Domain Description . . . 15
   2.2. Requirements . . . 36
   2.3. Implementation . . . 47
3. Analytic Part . . . 56
   3.1. Concept Analysis . . . 56
   3.2. Validation and Verification . . . 81
4. Specific Topics . . . 95
   4.1. Stakeholder Identification . . . 95
   4.2. Personas . . . 97
   4.3. Traceability . . . 101
   4.4. Domain Requirements Acquisition . . . 104
   4.5. Inconsistency Management . . . 106

1. Informative Part

1.1. Team

For Milestone 1, the Digital Home Inventory team is organized into feature-based groups to support parallel development: Core Inventory, Alerts & Restock, Sharing & Budget, and Reports & Insights; plus a cross-functional Design Team, a Documentation Team, and two Project Managers. In this documentation, all team members are considered partners because they have a direct stake in and responsibility for Milestone 1 outcomes (planning, design, implementation, testing, and documentation), not merely being end users of the system-to-be. No external partners have been identified for this milestone; if any arise, their roles and responsibilities will be documented in this section. Potential external partners may include a faculty mentor, domain informants (students interviewed), or external technical advisors; if involved beyond feedback (e.g., providing recurring reviews, requirements validation, or architectural guidance), their responsibilities and contributions will be recorded here. The lists are presented in alphabetical order and indicate each group's Team Lead. The Tasks column lists associated GitHub issue numbers (and may later include short notes about responsibilities as work completes).

Managers:

Student	GitHub username	Tasks
Kaysha L Pagan-Lopez	@Kay9876	
Luis J Cruz-Cruz	@LuisJCruz	

Documentation Team:

Student	GitHub username	Tasks
Alondra S. Arce Perea (Team Lead)	@alondra-arce	#13, #113, #150, #177 #233, #238, #244, #250
TBD (unassigned seat)		

Design:

Student	GitHub username	Tasks
Adriana Figueroa (Team Lead)	@AdrianaFigueroa9	#19, #22, #34, #51, #93, #94, #107, #176
Devlin Hahn	@devlin-hahn04	#38, #39, #41, #145
Jorge De León	@JorgeDeLeonOrama	#17, #116, #126, #127
Ronaldo Flores Nazario	@ronaldoflores9	#20, #42, #84
Eliel Hernandez Gonzalez	@eliel-hernandez	#25, #26, #153
Daniella Melero	@daniellameleroo	#40, #44, #45, #48
Natalia S Vera Rivera	@nataliavera6	#21, #102, #103, #104, #106

Student	GitHub username	Tasks
Ian Rivera	@Programian	#29, #115, #118, #141
Jayden Sánchez	@jaydensanchez12	#56, #129, #130

Team 1: Core Inventory:

Student	GitHub username	Tasks
Kevin Gómez (Team Lead)	@kevgom018	#11, #12, #14, #22, #34, #107, #166
Devlin Hahn	@devlin-hahn04	#38, #39, #41, #145
Jorge De León	@JorgeDeLeonOrama	#17, #116, #126, #127
Januel E. Torres Marquez	@jankii03	#52, #135, #136, #137
Kian-Robert Ramos Vega	@kian-robert	#67, #108, #109, #131
Jorge Rivera	@JorgeRivera94	#15, #122
Lucas Matos Datiz	@L-A-M-D	#82, #231, #249, #267

Team 2: Alerts & Restock:

Student	GitHub username	Tasks
Solimar Cruz Hernandez (Team Lead)	@Solimar-Cruz	#22, #30, #32, #33, #34, #107
Ronaldo Flores Nazario	@ronaldoflores9	#20, #42, #84
Eliel Hernandez Gonzalez	@eliel-hernandez	#25, #26, #153
Kemuel Cubero Rosario	@Kemuel05	#23, #146, #147, #148
Kevin Lara Rodríguez	@kevinlara1	#87, #205, #206, #207
Aryam Z Diaz Torres	@aryamdiaz	#24, #112, #180, #181
Reinaldo J. Martinez Morales	@reinaldo-martinez1	#27, #28, #119, #199

Team 3: Sharing & Budget:

Student	GitHub username	Tasks
Andrea Segarra (Team Lead)	@andreasegarra	#22, #34, #35, #107, #132, #134, #179
Daniella Melero	@daniellameleroo	#40, #44, #45, #48
Natalia S Vera Rivera	@nataliavera6	#21, #102, #103, #104, #106

Student	GitHub username	Tasks
Fabiola Torres	@FabiolaZTorres	#46, #95, #96, #101
Fernando Castro	@fernan-castro	#71, #168, #173, #198
Yamilette Alemany	@yamilette-alemany	#18, #58, #59, #60
Ernesto S. Soto Rivera	@ernestosoto3	#31, #98, #99, #100

Team 4: Reports & Insights:

Student	GitHub username	Tasks
Fabian Velez (Team Lead)	@FabianVelezOcasio	#22, #34, #107, #138, #139, #157, #178
Ian Rivera	@Programian	#29, #115, #118, #141
Jayden Sánchez	@jaydensanchez12	#56, #129, #130
Nicolás Rivera Batista	@nicolasrivera25	#16, #193, #196
Samarys Barreiro Melendez	@SamarysB	#36, #111, #113, #155, #156
Jorge Peralta	@PeraltaJorge9128	#43, #143, #159
Gerardo Ruiz	@Gerardoruiz1	#55, #201, #261, #263
Aliana Figueroa	@alianasofiaaa	

1.2. Current Situation, Needs, and Ideas

1.2.1. Current Situation

Students living independently often lose track of household items such as groceries, cleaning supplies, toiletries, and sometimes medicine. Tracking is usually ad-hoc (memory, notes, receipts, or messages), so information becomes outdated quickly. As a result, students may buy duplicates, run out of essentials unexpectedly, and sometimes discover expired items during cleanup.

In shared households, the problem is worse: roommates lack visibility into what is already available, which increases duplicate purchases and makes budgeting harder. In many student households there is no consistent shared method for tracking items across people and time; tracking is ad-hoc (memory, notes, messages), so information becomes stale quickly.

1.2.1.1. Domain context

Students and young adults living independently often struggle to keep track of household items, shared supplies, and personal belongings. While several home inventory applications exist, most are designed for insurance documentation or business inventory management rather than lightweight everyday use.

As a result, many users rely on memory, notes, or informal tracking methods, which leads to forgotten purchases, duplicate buying, and poor visibility into shared household resources. Most

tools students try (notes, basic lists, or heavier inventory apps) are either too informal to stay accurate over time or too cumbersome for everyday tracking, so shared inventory information quickly becomes unreliable.

1.2.1.2. Who is involved (actors)

- Primary actors: Students and young adults managing personal household items
- Secondary actors: Roommates or cohabitants sharing supplies
- Organizations/systems involved: Existing home inventory mobile applications (e.g., Sortly, PantryCheck)

1.2.1.3. Current workflow (as-is)

1. User purchases or owns household items
2. User optionally records items manually in notes or an inventory app
3. Inventory is rarely updated consistently over time
4. User forgets what items are already available
5. User repurchases duplicate or unnecessary items
6. In shared households, roommates lack visibility into available supplies

1.2.1.4. Pain points

Pain point	Who experiences it	Evidence / example
Too much of a hassle when adding items	Students and casual users	User reviews have reported that manual item entry consumes too much time and discourages the continued use of the software.
Inventory becomes outdated quickly	Almost all users	Many users abandon inventory apps after initial setup due to maintainance taking too much time.
Excessive or poorly timed notifications	Mobile users	Reviews indicate that too many notifications pop up in other applications, leading users to disabling alerts, one of the main purposes of said apps.
Pricing perceived as too high for casual use	Students and young adults	Premium tiers are often designed for business users rather than individuals, but lock features that could come in handy for individual users.

Pain point	Who experiences it	Evidence / example
Lack of effective shared household inventory support	Roommates and cohabitants	Most existing apps provide limited or no real-time collaboration features, which makes it difficult for multiple household members to maintain a single accurate inventory. This can lead to miscommunication or duplicate purchases for the house/dorm.

1.2.1.5. Consequences

- Time/cost impact: Users and students waste money on duplicate purchases and spend extra time checking inventory manually using written lists or other tracking software.
- Quality/accuracy impact: When tracking relies on tools that are abandoned or partially used (e.g., because advanced features are unavailable or too cumbersome), the record becomes incomplete and therefore untrusted.
- User experience impact: Frustration leads to abandonment of existing apps.
- Organizational impact: Shared households experience coordination inefficiencies.

1.2.1.6. Assumptions and constraints (as-is)

- Assumption: Target users prefer low-effort mobile workflows.
- Assumption: Users are more likely to maintain inventory if entry is fast and simple.
- Constraint: Users have limited patience for manual data entry.
- Constraint: The solution must work well on mobile devices with intermittent connectivity.

1.2.2. Needs

The needs below describe what students living independently and their roommates must be able to accomplish to manage household essentials reliably. These needs are expressed as outcomes and capabilities (what stakeholders need to achieve), independent of any specific system-to-be.

Needs by actor group

1.2.2.1. Actor group: Students and young adults

- Need N1: Minimize the effort and time required to keep track of household items
- Need N2: Maintain an accurate understanding of what items are available as they are consumed
- Need N3: Determine whether an item is already available before making a purchase

1.2.2.2. Actor group: Roommates / Cohabitants

- Need N4: Maintain a shared understanding of what items are available to the household
- Need N5: Coordinate responsibility for replenishing shared items to avoid duplicate or missed

purchases

1.2.2.3. Prioritized needs

ID	Need statement	Priority	Rationale
N1	Users must be able to record items in seconds	High	Addresses high-friction item entry reported in user feedback
N2	Users must maintain an up-to-date inventory with low ongoing effort	High	Mitigates abandonment caused by needing high maintainance
N3	Users must quickly verify existing items before purchasing	High	Reduces duplicate purchases and wasted spending
N4	Household members must share inventory visibility	Medium	Improves coordination in shared living environments
N5	Household members must coordinate replenishment of shared items to avoid duplicate or missed purchases	Medium	Reduces miscommunication and duplicate spending in shared living environments

1.2.2.4. Quality Needs (Non-Functional)

- Usability: Household members need to update item tracking with minimal effort and without training.
- Reliability: Household members need to trust that the tracked inventory matches what is actually available.
- Performance: Household members need inventory information quickly enough to use it while shopping or planning.
- Availability: Household members need tracking to remain usable even when connectivity is unreliable.
- Security/Privacy: Household members need confidence that household data is only visible to intended people.
- Scalability: Household members need the experience to remain smooth as more items, households, and members are added.

1.2.3. Ideas

Outline candidate solution directions. Ideas are not commitments; they are options to address the needs. The idea below focuses on solution directions and capabilities; implementation technology choices are deferred to later sections.

1.2.3.1. Lightweight Smart Inventory

- Core concept: A lightweight household inventory tool optimized for very fast item capture and shared visibility in student households, so records stay current with minimal effort.

- Needs addressed: N1, N2, N3, N4, N5
- Key capabilities:
  - Fast item capture with minimal information required at the moment of entry
  - Shared household view of what items are available and who last updated them
  - Quick search and filtering to check item availability
  - Optional prompts to review items that may be running low or expiring (kept low-noise)
  - Works during temporary connectivity gaps and reconciles changes when connectivity returns
- Expected benefits:
  - Reduces the hassle of item entry
  - Improves long-term inventory accuracy
  - Prevents duplicate purchases
  - Enables smooth roommate coordination
  - Reduces the chance that people disable reminders by keeping prompts minimal and relevant
- Risks / unknowns:
  - Users may still abandon after initial novelty wears off
  - Conflicting updates when multiple household members change the same item list around the same time
  - Temporary connectivity gaps may lead to mismatched inventories until updates reconcile
  - User adoption of shared features may vary
- How we will validate this idea:
  - Run quick usability sessions to see whether people can capture an item fast enough during realistic moments (e.g., after shopping)
  - Track weekly retention during pilot testing
  - Test with small shared households (2-4 users)
  - Observe whether optional prompts are perceived as helpful or disruptive (and whether people disable them)

1.3. Scope, Span, and Synopsis

1.3.1. Scope and Span

Scope

The project's scope is to carry out a software engineering project in the domain of student household management. The project will investigate how students living independently (alone or with roommates) keep track of household items and spending, and it will produce a system-to-be plus the supporting deliverables needed to build it responsibly (domain description, requirements,

architecture/design, implementation, validation/verification, and documentation). The project targets common pain points caused by relying on memory and informal tracking, and it will support both individual and shared-household workflows. The project will encompass several key areas:

- Domain Engineering: Analyze how students currently manage household items, including common breakdowns such as forgetting what is available, inconsistent updates, and poor roommate coordination. Identify the major actors (students living alone, students living with roommates/cohabitants) and the domain concepts (items, categories, locations/rooms, quantities, thresholds, expiration dates, shared responsibility).
- Requirements Engineering: Define system requirements that support inventory accuracy, fast item updates, and coordination. Requirements will cover core inventory maintenance, item organization, information lookup before purchasing, shared-household coordination, and budget awareness, while also capturing constraints such as busy schedules, low tolerance for complex setup, and the need for privacy controls.
- Software Architecture: Develop a high-level architecture that supports mobile use, persistent storage, and shared-household data synchronization. The architecture will identify major components and their responsibilities (client, data storage, synchronization, authentication/authorization, and notifications/reminders) and will be designed to support testability and maintainability.
- Software Design: Design workflows that keep inventory maintenance low-friction (capture, update, lookup) and that remain understandable in shared households. Provide clear feedback when items are low, near-expiration, or shared, and present budget summaries in a way that supports planning rather than reactive purchasing.
- Software Process: The project will follow an Agile, feature-team structure. Each feature team owns its feature end-to-end (research, design collaboration, implementation, testing, and documentation). Work will be tracked and reviewed using a structured issue and pull-request workflow to keep changes traceable and reviewed. Continuous feedback from team walkthroughs and scenario reviews will guide refinement of requirements and UX decisions.

Span

The project's span focuses on typical student living environments (dorms, apartments, shared houses) where household management is informal and time-constrained. The initial span targets small households with a modest inventory and a small number of cohabitants, and it assumes frequent lightweight updates rather than occasional heavy maintenance.

- Specifics of the System: Users can maintain a centralized inventory organized by room and category, update item quantities, and optionally track expiration dates and priorities. Household members can choose how they want to be informed about low-stock and upcoming expirations, like low-stock and upcoming expirations. Shared households can view a common inventory and distinguish between shared and personal items to reduce duplicates and conflict. Budget tracking connects purchases to categories and provides basic spending summaries and reports.
- Target Audience and Expansion: The initial span targets students living independently, including those living with roommates. Expansion may include richer analytics (usage rates and trends), more advanced report types, optional integrations with external productivity or budgeting tools, and improved automation for restock planning. Any expansion will remain

centered on student needs and low-friction maintenance.

- Methodology and Maintenance: The project will proceed iteratively, refining documentation and implementation through validation and verification activities. Progress will be supported through iterative cycles of documentation, implementation, and validation/verification activities, refining scope and requirements as stakeholder feedback and early testing reveal gaps. The cycle of feedback and refinement will ensure the system remains practical for real student workflows and constraints.

1.3.2. Synopsis

This project addresses inefficiencies and stress in student household management that arise when tracking of essentials relies on memory or informal methods. The project will produce a system-to-be that supports lightweight tracking of household items and spending for students living alone or with roommates, with particular focus on minimizing maintenance effort, reducing duplicate purchases, and lowering the likelihood of items expiring unnoticed. Shared-household use will be supported so that cohabitants can coordinate around what is available and what needs replenishing.

Work will proceed through domain engineering and requirements engineering (including personas and scenarios), followed by architectural and design decisions and incremental implementation. Validation and verification activities (walkthroughs, scenario-based reviews, and testing) will be used to confirm that the evolving design matches stakeholder needs and that key behaviors work reliably. Across milestones, the team will iteratively refine the documentation and implementation based on evidence and feedback, aiming for an approach that remains practical for real student workflows over time.

1.4. Foundational Activities for System Development

A successful system-to-be is built on more than just code. The following activities establish the foundation upon which the Digital Home Inventory system will be developed, ensuring it accurately reflects student needs and performs reliably in real-world conditions.

1.4.1. Domain and Requirements Engineering

Learning how students actually live comes first. Through conversations and watching how people manage their daily routines—like using shared notes, talking things over with roommates, or simply trying to remember what they need—the project builds a picture of current habits that exist without any special application. Some students live alone, others share space with several people. Some organize carefully, others wing it. From these observations, the team will extract domain concepts (e.g., item types, locations, shared responsibility, replenishment routines, and expiration awareness) and translate them into stakeholder-centered needs, personas, and scenarios. These will then be refined into a first set of user stories and measurable requirements, with open questions explicitly recorded for later research. This reduces the risk of building requirements that do not match real student routines. To build this understanding, the team will use short interviews and guided walkthroughs of students' current practices, collect a small set of concrete "day-in-the-life" scenarios, and review feedback from existing inventory/list tools to identify recurring friction points.

1.4.2. Documentation and Knowledge Management

As the project grows, decisions about the application should not live only in people's heads or get buried in old messages. Writing things down in one place gives everyone access to the same information. Keeping documentation versioned and maintained alongside development work ensures it evolves with the system and remains easy to review and update. When someone joins later, they can read about why certain choices were made and how different pieces fit together. The same record helps keep everyone using the same words for the same things—whether "pantry" means only food or includes cleaning supplies, for example. Without this habit, people start remembering things differently or making assumptions that others do not share. The project becomes harder to keep consistent as new features get added over time.

1.4.3. Quality Assurance Planning

An application that surprises users or fails when they need it will not become part of anyone's routine. Knowing when a feature is truly finished means agreeing on what success looks like beforehand. In addition to scenario-based validation, the team will plan verification activities such as unit/integration tests for core behaviors and periodic walkthroughs to check consistency between terminology, requirements, and implementation. For alerts, success might mean a student finds out about expiring yogurt in time to eat it. For the shared list, it might mean roommates see the same information when deciding who goes to the store. Walking through made-up but realistic situations with actual students helps catch mismatches between what the team builds and what users actually need. Talking through examples together—like a week in the life of someone trying to stick to a grocery budget—shows whether the planned features will hold up in real life. The point is an application that students find genuinely helpful, not one that simply passes technical checks.

1.4.4. Project Management and Coordination

Several groups working on different concerns of the same system-to-be need to stay connected. Regular conversations between teams working on inventory, alerts, budgeting, and reports help spot potential trouble before it slows everyone down. A shared work-tracking board will make ownership and progress visible and help the team manage dependencies between features. Looking over each other's work before it becomes part of the main application catches small issues early. When the team working on reports needs information about spending, checking in with the budgeting team early saves redoing work later. These habits of talking and coordinating mean that when it comes time to bring everything together, the pieces fit without surprises.

1.5. Derived Goals

1.5.1. Primary Derived Goals

In addition to the primary goal already established by the current situation and needs, this project pursues several secondary outcomes that guide decision-making across milestones. A key derived goal is to encourage more sustainable household routines among students by reducing avoidable waste over time—such as items expiring unused or being repurchased unnecessarily—through better day-to-day awareness of what is already on hand. Another derived goal is to improve adoption and continued use: the project should support habits that remain realistic under busy student schedules, so the practice of keeping track of essentials does not collapse after an initial

setup period.

A further derived goal is to strengthen coordination and trust in shared households. Even when people share expenses and supplies, misunderstandings about what is available and who will replenish items can create friction; the project aims to reduce these coordination breakdowns as a secondary social outcome of improved household awareness. Where budget awareness is in scope, an additional derived goal is to help students reflect on spending patterns around household essentials, supporting more informed planning rather than reactive purchases.

Finally, the project has internal quality goals that support sustainable progress across milestones. The team aims to maintain stable terminology and consistent documentation so decisions remain understandable and traceable over time, reducing rework and concept drift. The project will also emphasize evidence-based development by using validation and verification activities—such as personas, scenarios, walkthroughs, and testing—to continually check that the team's understanding matches stakeholder reality and that key behaviors work as intended.

1.5.2. Competing Goals and Resolutions

Although the project's derived goals reinforce one another in many ways, some of them also create natural tensions that must be handled explicitly. Identifying these tensions helps the team avoid inconsistent design decisions and produces derived requirements that clarify how the system should behave when important goals compete.

1.5.2.1. 1. Low-effort use vs. inventory accuracy and reliability

Competing goal A: Keep the system lightweight enough that students can realistically maintain it under busy schedules.
Competing goal B: Preserve inventory information with enough accuracy and completeness for reminders, shared coordination, and planning to remain trustworthy.

These goals compete because reducing friction often means reducing the amount of information required at entry time, but requiring too little information can make the inventory incomplete, stale, or unreliable. The project already emphasizes both low-effort maintenance and accurate inventory visibility, so the system must balance continued adoption with dependable data quality.

Resolution policy: The system should minimize required user effort during common workflows, but it must still collect and preserve the minimum information needed to support accurate inventory state and reliable system behavior.

Derived requirements:

- `DG-REQ-01` Users shall be able to create an initial item record with only the minimum information needed to identify and track that item, while adding richer metadata later as needed.
- `DG-REQ-02` Low-stock and expiration-related behaviors shall only occur when the item information required to evaluate those conditions is present and valid.
- `DG-REQ-03` Recorded item metadata shall remain stable and trustworthy over time unless an authorized user explicitly changes or removes it.

- `DG-REQ-04` Routine item updates shall remain lightweight enough to fit ordinary student workflows without creating excessive maintenance burden.

Trace hooks: `N1`, `N2`, `N3`; derived goal for adoption and continued use; reliability quality need; `US-INV-01`, `US-INV-02`.

1.5.2.2. 2. Timely reminders vs. avoiding notification fatigue

Competing goal A: Ensure users are reminded in time about low-stock and expiration conditions.
Competing goal B: Avoid excessive or poorly timed alerts that cause frustration, desensitization, or notification disablement.

These goals compete because a highly proactive reminder strategy may reduce the chance that important conditions are missed, but it also risks overwhelming users and undermining trust in the system. The project goals explicitly include automated low-stock reminders, expiration alerts, and customizable thresholds, while also emphasizing that notifications should remain helpful and relevant.

Resolution policy: The system should favor timely reminders for important conditions, but it must rate-limit and deduplicate recurring alerts so persistent states remain visible without becoming spam.

Derived requirements:

- `DG-REQ-05` Users shall receive low-stock reminders often enough to support timely replenishment, but not more than once per item for the same household member group within a single calendar day.
- `DG-REQ-06` Users shall receive expiration-related reminders often enough to support timely action, but not more than once per item for the same household member group within a single calendar day unless a new alert condition is created by a state change.
- `DG-REQ-07` Repeated sweep executions shall not produce duplicate notifications for the same item-condition pair within the same daily reminder period.
- `DG-REQ-08` Reminder behavior shall favor relevance and timeliness over notification volume by limiting prompts to conditions supported by valid item data and active alert rules.

Trace hooks: `N2`; usability and reliability quality needs; `Lightweight Smart Inventory`; `US-ALT-01`.

1.5.2.3. 3. Shared household visibility vs. privacy and least privilege

Competing goal A: Increase household coordination by making shared inventory visible to cohabitants.
Competing goal B: Protect user-specific or sensitive information by exposing only the information necessary for coordination.

These goals compete because coordination improves when household members can see what is available and what needs replenishment, but excessive visibility can expose ownership, spending, or personal-item information that not everyone should access. The project scope includes both shared inventory coordination and privacy/security needs, so the system must support collaboration without ignoring access boundaries.

Resolution policy: The system should expose the minimum shared information required for household coordination while restricting user-specific metadata to the appropriate scope and permissions.

Derived requirements:

- `DG-REQ-09` Household members shall be able to see the availability and replenishment-relevant status of shared items.
- `DG-REQ-10` Shared household data and personal or user-specific item data shall remain distinguishable when both exist in the same household context.
- `DG-REQ-11` Ownership, spending, and other user-specific item information shall only be visible to users with permission to view it.
- `DG-REQ-12` The distinction between shared items and personal items shall remain clear whenever that distinction affects replenishment or household coordination.

Trace hooks: `N4`, `N5`; derived goal for shared-household coordination and trust; security/privacy quality need; `US-SB-01`.

1.5.2.4. Summary of trade-off handling

These competing-goal resolutions clarify that the system must not optimize convenience, reminders, or transparency in isolation. Instead, the project balances adoption, data trustworthiness, alert usefulness, and household coordination by introducing derived requirements that are concrete enough to guide implementation and later validation and verification.

2. Descriptive Part

2.1. Domain Description

2.1.1. Domain Rough Sketch

Raw, unprocessed domain notes and examples. Organize by feature team if possible.

Please note that most(if not all) of the quotes mentioned have been translated from Spanish. Also names have been removed to protect privacy, in their place, the person's perceived role in the domain is shown (e.g. instead of Kevin Gomez, we would place Student).

2.1.1.1. Interview Snippets and Raw Observations

Shared Apartment - 3 Students (Interview, Jan 20 2026)

- Student A: "If it is not on our fridge note, we assume we are out."
- Student B: "Sometimes I check the pantry before leaving, but by the time I get to Walmart I forgot what we needed."
- Student C: "We have rice in two places. One bag in the kitchen cabinet and another in a plastic bin in the hallway."
- They currently use: fridge sticky note + WhatsApp group chat.
- No one is formally responsible for keeping track of quantities.
- Milk is usually bought twice in the same week at least once per month.
- When asked how they know they are 'low': answer was "when it feels light" or "when we open the backup."
- They do not write down when the backup is opened.

Student Living Alone (Interview, Feb 2 2026)

- "I buy in bulk from Costco when I go with my parents."
- Mentions buying 24 yogurt packs at once.
- Yogurts sometimes expire; estimated 3-5 thrown away last semester.
- Keeps cleaning supplies in bathroom closet and kitchen under sink.
- Has never counted exact number of items; estimates visually.
- Uses Notes app occasionally but stops after 1-2 weeks.
- Said: "If it takes more than a few seconds to log, I won't do it."

Household Owner (2 adults, 1 child)

- Owner invites family members to shared grocery list app.
- One adult does most purchases but child sometimes uses last item without saying.

- Example: toilet paper runs out unexpectedly when guests visit.
- Keeps medicines in a separate drawer; expiration dates important.
- Has thrown away expired medicine without realizing there was a second unopened box.
- Wants to be able to remove access when someone moves out.

Student Athlete (Interview, Feb 2 2026)

- "During exam weeks I eat way more snacks."
- Protein bars may last 2 weeks normally, but 4-5 days during midterms.
- Does not adjust shopping quantity formally; just reacts when gone.
- Buys based on memory of how fast it ran out "last time."

Observations from Informal Brainstorming Session

- People describe items as "full", "almost empty", "backup", rarely with numbers.
- Essential items mentioned repeatedly: toilet paper, soap, toothpaste, milk, rice.
- Perishables mentioned: yogurt, vegetables, chicken, medicine.
- Bulk buying motivated by lower price per unit.
- Waste mentioned mainly with food, not cleaning supplies.
- Duplicate buying most common with milk, eggs, rice.
- Some people separate items by room (kitchen, bathroom, laundry), others do not.
- No participant mentioned doing periodic full inventory checks.
- All participants said they have at some point "forgotten what they already had at home."
- Some participants said they prefer taking a picture of the pantry instead of writing a list.

Edge Situations Mentioned

- Barcode does not scan (damaged packaging).
- Expiration date unreadable or missing.
- Guests consume items faster than usual.
- Someone moves out and still has access to shared list.
- Item bought in bulk has multiple different expiration dates in same package.
- Backup stored in car trunk and forgotten.

2.1.2. Terminology

This section defines the domain’s ubiquitous language: preferred terms for entities, actions, events, and rules observed in student households. The definitions aim to stay independent of any specific system-to-be or implementation.

Term	Type	Definition	Synonyms / notes
Household	Entity	A living unit (dorm, apartment, shared house) where one or more people manage household essentials.	Previously “Home”.
Household member	Entity	A person who belongs to a household and participates in consuming, replenishing, or coordinating around items.	Previously “Owner” or “Occupant”.
Location	Entity	A storage place within a household (kitchen cabinet, pantry area, bathroom closet, car trunk, hallway bin).	Previously “Room”.
Item type	Entity	The kind of item independent of a specific package (e.g., “Milk”, “Dish soap”, “Toilet paper”).	Previously “Product”.
Item instance (package)	Entity	A concrete package/unit of an item type (e.g., a specific milk carton or a specific box of medicine).	Previously “Stock”.
Inventory (reality)	Entity	The set of item instances that physically exist in the household at a given time.	Real world, not a list.
Tracked inventory record	Artifact	A representation of availability used by household members (fridge note, list, memory, chat message). It may diverge from reality.	Supports “tracked vs reality”.
Availability	Property	A household’s current understanding of “how much is left,” often qualitative (“full”, “almost empty”, “backup”).	May later be represented as quantity.
Quantity	Property	A more precise measure of availability (count/units). Quantity can be unknown or approximate.	If used, it must never be negative.
Low-stock judgment	Event/Rule	A decision that an item is running low, based on a household strategy (“feels light”, “opened backup”) or a threshold.	Not inherently numeric.
Low-stock threshold	Rule/Property	A boundary value used to decide when an item is considered low.	May be fixed quantity initially.
Out-of-stock	State/Event	The condition/event where an item is considered unavailable because it is gone (quantity at zero or judged empty).	Distinct from low-stock.
Backup supply	Practice/Concept	A household practice of keeping a spare instance/package (backup) to reduce the risk of running out.	Often used instead of numeric tracking.
Consumption	Action	Using an item over time, reducing availability.	May vary by context (exam weeks, guests).

Term	Type	Definition	Synonyms / notes
Replenishment (restock)	Action	Acquiring more of an item, increasing availability.	Often triggered after low/out/expiration.
Restock list / planner	Artifact	A household-oriented list of items that should be replenished soon.	Exists as a concept (notes/list).
Alert / notification (domain-level)	Artifact/Event	A message that informs a household member about an actionable condition (low-stock, out-of-stock, expiring/expired, budget threshold crossed).	Delivery method is an implementation choice.
Snooze / defer	Action	A household member delays acting on an alert for a period of time.	Domain intent: “remind me later”.
Dismiss / ignore	Action	A household member acknowledges an alert and chooses not to act (at least for now).	Should not cause repeated spam without changes.
Expiration date	Property	A date after which an item instance is considered expired/undesirable (applies to some items, not all).	May be unreadable/missing.
Near-expiry	State/Event	An item instance is approaching expiration within a defined window (e.g., N days).	Window definition belongs in requirements later.
Expired	State/Event	An item instance is past its expiration date.	Expired does not always imply “unavailable”; policy varies.
Spending event / spending entry	Record	A record of a financial transaction in a household (purchase/refund/adjustment), optionally linked to an item type and category.	Supports budgeting/reporting.
Spending category	Entity/Label	A grouping for spending (e.g., groceries, cleaning, personal care).	May align with item categories, but not required.
Budget	Entity/Rule	A financial limit defined for a period (household-level or category-level) used to evaluate spending.	Budget window/cycle is part of requirements.
Budget cycle (period)	Concept	The time window over which spending is compared to a budget (weekly, monthly, custom).	Resets/rollovers are policy decisions.
Budget threshold crossing	Event	Spending reaches or exceeds a defined threshold (e.g., 80%, 100%).	Can trigger alerts, subject to anti-spam rules.

Term	Type	Definition	Synonyms / notes
Transaction (quantity change)	Record	A record of an availability change (consume/restock/adjust) with time and reason.	Supports traceability and verification.

2.1.3. Narrative

A cohesive story of how the domain works, independent of any specific software system.

2.1.3.1. Overview

Students living independently go through a continuous cycle of acquiring, using, and replacing household items. They buy food and supplies when they have time and money, store them in locations within their living space, and consume them gradually. When items run low or run out, someone must notice and take action to replenish them. In shared households, this cycle involves multiple household members who must coordinate—or fail to coordinate—about what is needed, who will buy it, and when.

Tracking is typically informal. Many households rely on memory, a fridge note, a chat message, or a quick pantry check. This tracked understanding can diverge from reality: items may exist but be forgotten in a different location, backups may be opened without being recorded, and purchases may be duplicated because household members act on incomplete information. Waste can occur when items expire unnoticed or when duplicates are purchased while another instance is already available.

Consumption patterns are not stable. During exam weeks or when guests visit, items can be used faster than usual. Bulk purchases can increase the number of item instances in the household at once, sometimes with different expiration dates within a single bulk pack. The household cycle repeats constantly under constraints of limited time, attention, and coordination.

Some households also keep an informal understanding of spending on essentials. Purchases may be remembered loosely, estimated from receipts, or discussed among roommates when money feels tight. When spending is tracked (even imperfectly), it influences decisions such as buying in bulk for lower unit price, delaying non-urgent restocks, or choosing cheaper alternatives. In shared households, disagreements can arise when it is unclear which purchases are personal versus shared or when one member repeatedly pays for shared items. Budget awareness therefore becomes another dimension of household coordination, alongside item availability and replenishment.

2.1.3.2. Actors and artifacts

Actors (domain roles): * Students living alone (single-member household) * Students living with roommates/cohabitants (multi-member household) * Household members who often coordinate replenishment (informal “household manager” role) * Household members who focus on budget awareness (budget-conscious role)

Artifacts used today (domain-visible): * Fridge notes / whiteboards * Chat messages (e.g., group chat) * Notes app lists * Pantry photos * Receipts or memory-based estimates

2.1.3.3. Typical domain flow (as observed)

1. Household members acquire items (sometimes in bulk) and store them in household locations.
2. Items are consumed over time; availability decreases.
3. A household member checks availability using informal artifacts (note, memory, pantry check).
4. If the member judges an item to be low or out (often using qualitative strategies like “feels light” or “opened backup”), replenishment is triggered.
5. In shared households, coordination may succeed (one person buys) or fail (duplicate buying).
6. Some items expire before use; households may discover this late and discard them.
7. Household members may keep informal awareness of spending; when tracked, spending influences replenishment choices and can create shared-vs-personal disputes.
8. Membership changes (move-out) require the household to update who participates and who is informed.

2.1.4. Entities

Key domain entities (nouns) and their attributes. These entities describe the domain independently of any specific system implementation.

Entity	Description	Key attributes
Home	A physical space where people live and where a household inventory is managed.	name, rooms, inventory, stakeholders
Room	A specific area within a Home where Stock can be stored (e.g., kitchen, bedroom, pantry, garage).	name, purpose (optional)
Product	A household item definition that is commonly restocked (e.g., milk, toilet paper, dish soap). This represents the item concept independent of a specific purchase.	name, description, tags, unit (optional), image (optional)
Stock	An instance of a Product that represents a specific purchased item tracked in the inventory (e.g., two cartons of milk tracked separately by expiration date and location).	brand, quantity, status, expiration date (optional)
Inventory	The collection of Products and Stock in a Home, representing what is currently available.	reference to owner, stock registry
Stock Location	The association between a Stock item and the Room where it is stored.	stock, room

Entity	Description	Key attributes
Stakeholder	A person who has a vested interest in a Home and its Inventory. Stakeholders interact with the system according to assigned Roles.	name, associated home(s), role(s)
Owner	A Stakeholder role representing the primary manager of a Home who has the highest access and permission to make changes.	permissions, managed home(s)
Member	A Stakeholder role representing a person who lives in a Home and should have access to interact with its Inventory.	permissions, associated home(s)
Guest	A Stakeholder role representing a person who has view-only or limited access to a Home and its Inventory.	permissions, associated home(s)
Category	A grouping label used to organize Products by type or purpose (e.g., food, cleaning, electronics).	name, description (optional)
Shopping List	A temporary list of Products that need to be purchased soon.	list of products, quantities (optional), priority (optional)
Supplier	A store or provider where a Product can be purchased or restocked (e.g., Walmart, Costco, CVS).	name, location (optional), notes (optional)
Budget	A plan that allocates money across Categories to support household purchasing decisions.	amount, categories, time period (optional), limits (optional)
Household	A living unit where household essentials and spending are managed.	name (optional label), members, locations, inventory (reality), optional budgets
Household member	A person who belongs to a household and consumes/replenishes items.	name/identifier (domain-level), household reference, role (informal)
Location	A storage place within a household where items are kept.	name/label, household reference, optional purpose
Item type	A definition of an item independent of a specific package/purchase.	name, optional category, optional brand/size/variant, optional barcode
Item instance (package)	A concrete package/unit of an item type that exists in the household.	item type reference, location reference, ownership (personal/shared), quantity/availability, optional purchase date, optional expiration date, optional notes

Entity	Description	Key attributes
Tracked inventory record	A representation households use to reason about availability (may diverge from reality).	artifact type (note/chat/list/memory), item reference (explicit or implicit), last-updated (if recorded)
Restock list/planner	A list of items judged to need replenishment soon.	entries (item types), reason (low/out/expired), optional responsibility/intent marker, status (active/deferred)
Alert / notification record	A record of an alert-worthy condition presented to household members.	type (low/out/near-expiry/expired/budget), target (item type/instance or budget), created time, read/unread, dismissed/deferred
Spending entry	A logged financial transaction event within a household.	amount, date/time, payer (member), scope (shared/personal), optional item type reference, category, notes, transaction type (purchase/refund/adjustment)
Budget	A spending limit defined for a period.	scope (household/category), limit amount, cycle (period), start/end (or recurrence), thresholds (warning levels), visibility rules
Budget status (derived)	A derived view of spending vs budget within a cycle.	spent-to-date, remaining, percent used, threshold-crossed flags

2.1.5. Events

Domain events are things that happen in the domain that matter to stakeholders (often phrased as past-tense verbs). These events are independent of any particular system-to-be, even if a later system represents or reacts to them.

Event	Description	Notes / examples
Item acquired	A household member obtains an item instance (possibly in bulk) and brings it into the household.	“Bought 24 yogurts from Costco.”
Item stored	An item instance is placed in a household location.	“Rice stored in kitchen cabinet; backup stored in hallway bin.”
Item consumed	A household member uses an item, decreasing its availability.	Consumption rate may change during exams/guests.
Backup opened	A household member begins using the backup supply of an item.	Often used as a qualitative low-stock signal.
Low-stock judged	A household member judges an item as running low.	“Feels light” / “backup opened” / threshold-based.

Event	Description	Notes / examples
Out-of-stock noticed	A household member discovers an item is gone when needed.	“Toilet paper runs out unexpectedly.”
Replenishment decided	A household member decides the household should acquire more of an item type.	May be prompted by low/out/expiration.
Item replenished	New item instance(s) of an item type are acquired to restore availability.	Can be duplicated if coordination fails.
Duplicate purchase occurs	More of an item type is acquired unnecessarily because members acted on inconsistent information.	Common for milk/eggs/rice per rough sketch.
Item approaching expiration	An item instance is within a near-expiry window (window definition later).	Motivates timely decisions.
Item expired	An item instance passes its expiration date (if it has one).	Policy varies by item type/household.
Item discarded	A household member throws away an item instance (expired/spoiled/unwanted).	Can occur even when another unopened instance exists.
Alert generated (domain-level)	A low/out/near-expiry/expired/budget condition becomes actionable and is communicated to household members.	Delivery method is not part of the domain event.
Alert dismissed / deferred	A household member acknowledges an alert and chooses to ignore it or act later.	Supports “snooze” and anti-spam behavior.
Restock entry created/updated	An item is added to (or updated on) a restock list/planner due to low/out/expired conditions.	May be manual or derived from judgments.
Spending recorded	A household member logs a spending transaction (purchase/refund/adjustment).	May be shared or personal.
Budget threshold crossed	Spending in a budget cycle reaches a warning level or exceeds the limit.	May trigger an alert.
Budget cycle started/ended	A new budget period begins or a period closes.	Supports reset/rollover policies.
Household membership changed	A member moves in/out; household participation changes.	Motivates access removal expectations later.

2.1.6. Functions

Major domain functions/capabilities (what can be done in the domain), described independently of

UI. Function signatures are illustrative and may evolve.

Function	Description	Signature (illustrative)
Add item instance	Introduce a new item instance/package into a household's tracked representation.	addItemInstance : Household >< ItemType >< Location >< Quantity? >< ExpirationDate? → Household
Move item instance	Change where an item instance is stored.	moveItemInstance : Household >< ItemInstance >< Location → Household
Record consumption	Decrease availability of an item instance (or item type record) due to use.	consume : Household >< ItemInstance >< QuantityDelta → Household
Record replenishment	Increase availability by adding new item instance(s) or increasing quantity.	restock : Household >< ItemType >< Location >< QuantityDelta >< ExpirationDate? → Household
Judge low-stock	Determine whether an item type is considered low based on household strategy/threshold.	isLowStock : ItemType >< Availability >< Threshold? → Bool
Determine expiration status	Classify an item instance's expiration state when an expiration date exists.	expirationStatus : ItemInstance >< DateNow → (Active
NearExpiry	Expired	Unknown)
Create restock entry	Add an item type to the restock list/planner with a reason.	addToRestockList : RestockList >< ItemType >< Reason → RestockList
Defer / dismiss restock entry (manual override)	Allow a household member to postpone or remove an entry from restock planning.	deferRestock : RestockList >< ItemType >< Until? → RestockList
Generate alert (domain-level)	Create an alert record when an actionable state is reached (low/out/near-expiry/expired/budget threshold).	generateAlert : AlertLog >< AlertType >< Target → AlertLog
Acknowledge alert (dismiss/defer)	Record a household member's decision to dismiss or defer an alert.	ackAlert : AlertLog >< Alert >< Action(dismiss|defer) >< Until? → AlertLog
Record quantity change transaction (traceability)	Append a transaction describing why/when availability changed.	recordTransaction : TransactionLog >< ItemInstance >< Change >< Reason >< Time → TransactionLog
Record spending event	Create a spending entry capturing who paid, how much, when, and whether it is shared or personal.	recordSpending : Household >< Amount >< Date >< Payer >< Scope(shared|personal) >< ItemType? >< Category? → Household

Function	Description	Signature (illustrative)
Adjust spending entry (audit-safe)	Correct spending records via adjustment/refund entries rather than destructive edits.	adjustSpending : Household >< SpendingEntry >< Adjustment → Household
Compute budget status	Compute spent-to-date and remaining budget for a household or category within a budget cycle.	budgetStatus : Budget >< List SpendingEntry >< DateNow → BudgetStatus
Detect budget threshold crossing	Determine whether spending crosses warning/limit thresholds.	budgetThresholdCrossed : BudgetStatus >< Thresholds → Bool

2.1.7. Domain Logic for Spending Tracking

2.1.7.1. Objective

This document defines the domain logic governing how spending events are recorded, attributed, and maintained within a shared household inventory system.

The purpose of this document is to establish consistent behavioral rules for financial tracking within shared environments. These rules provide the conceptual foundation necessary to support budgeting, reporting, and financial accountability features.

The logic defined here is independent of implementation and serves as guidance for schema design and future budgeting mechanisms.

2.1.7.2. Definition of Spending Event

A Spending Event represents a financial action associated with an inventory item within a shared household context.

A spending event captures:

- who paid
- what item was purchased
- the monetary amount
- when the transaction occurred
- whether the cost is individual or shared

Spending events represent historical financial records and must preserve traceability.

2.1.7.3. Core Spending Logic Rules

Rule 1: Spending is recorded only when a financial transaction occurs

Spending is created when a purchase is logged with a defined cost.

Implications:

- Creating an item without a cost does not generate spending.
- Updating an item to include cost generates a spending event.
- Editing non financial item attributes does not affect spending.

Conceptual relationship impact: Spending depends on a financial attribute of an Item.

Rule 2: Spending is attributed within an inventory context

All spending must be scoped to a specific inventory (household).

Implications:

- Spending cannot exist outside an inventory.
- Financial accountability is contextual to a shared environment.
- Spending must reference both Item and Inventory.

Conceptual relationship impact: Spending → Inventory (required association)

Rule 3: Spending must identify a payer

Every spending event must define the user responsible for the payment.

Implications:

- Anonymous financial records are not allowed.
- The payer must be a valid member of the inventory.
- Financial accountability is traceable to a participant.

Conceptual relationship impact: Spending depends on Membership.

Rule 4: Spending attribution may be individual or shared

Spending must define whether the cost applies:

- exclusively to the payer (individual spending), or
- to the shared household (shared spending)

Implications:

- Shared cost does not remove payer identification.
- Cost distribution logic is separate from spending creation.
- Attribution model must be explicit.

Conceptual relationship impact: Spending contains attribution state.

Rule 5: Spending events are historical and immutable

Spending records represent financial history and must preserve integrity.

Implications:

- Deleting an item does not delete associated spending.
- Users leaving the inventory do not erase past spending.
- Financial history must remain structurally consistent.

Conceptual relationship impact: Spending persists independently of Item lifecycle changes.

Rule 6: Spending cannot exist without a valid item reference

Spending is a dependent domain concept.

Implications:

- A spending event must reference an item that existed at the time of the transaction.
- Invalid or orphaned financial records are not allowed.
- If an item is soft-deleted, the relationship must remain traceable.

Conceptual relationship impact: Spending depends on Item existence.

Rule 7: Spending modification must preserve traceability

If a cost is corrected or updated, the system must preserve financial traceability.

Implications:

- Either a new spending event is created, or
- A modification log must exist.

Financial records must never silently overwrite historical values.

2.1.7.4. Role of Spending Logic in Budget Design

These rules establish the conceptual constraints that the budgeting model must satisfy. They define:

- how financial activity enters the system
- how spending relates to inventory structure
- how accountability is enforced
- how financial history is preserved

The logic defined here provides the foundation for:

- Budget Rules
- Spending Schema Fields
- Cost allocation models
- Reporting and analytics systems

2.1.8. Behaviors

Recurring behaviors or rules in the domain: patterns, policies, and "how things are done."

2.1.8.1. Behavior patterns

- **Restock cycle:** A stakeholder consumes stock → quantity decreases → low-stock threshold is crossed → alert issued → purchase is made → quantity replenished.
- **Shared purchase flow:** A stakeholder buys a shared item → logs the purchase → the system splits cost across household members → each member sees their share.
- **Expiration monitoring:** Stock nears its expiration date → the system transitions the item to an *Expiring* state → alert issued → stakeholder removes or discards the item → a waste event is logged.

2.1.8.2. Domain rules

- One barcode corresponds to exactly one Product definition (DR-1, DR-14, DR-17).
- A Stock duplicate entry submitted within 30 seconds triggers automatic quantity aggregation rather than a new entry (DR-7, IR-11).
- Spend Events are immutable — inventory corrections must never overwrite financial history (DR-3, DR-9).
- A Household must always have exactly one Owner (DR-4).
- Ownership must be transferred before an Owner can leave a Household (DR-4).

2.1.8.3. Edge cases

- If a stock entry is submitted while offline and synced later, the 30-second duplicate window is evaluated against the server-received timestamp, not the client timestamp.
- If an item has no expiration date, expiration monitoring is skipped for that stock entry.
- If an Owner attempts to leave without transferring ownership, the action is blocked until a new Owner is designated.

2.1.9. Inventory Sharing Flowchart

Author: Daniella Melero Pereira

2.1.9.1. Overview

This document explains the sharing workflow used in the Home Inventory. The sharing feature allows users to invite roommates or household members to access and collaborate on a shared inventory.

The flowchart provides a visual representation of how invitations are sent, processed, and how access to shared inventories is granted and managed.

2.1.9.2. Sharing Workflow Summary

The sharing process follows these main steps:

1. The inventory owner selects an inventory to share.
2. The owner sends an invitation to a roommate.
3. The system marks the invitation as **Pending**.
4. The roommate receives the invitation.
5. The roommate can accept or reject the invitation.
6. If accepted, access to the shared inventory is granted.
7. If rejected, the invitation is closed.
8. The owner can manage or revoke access at any time.

2.1.9.3. Flow States

The sharing process includes the following states:

- **Pending** — invitation has been sent but not yet accepted.
- **Accepted** — roommate accepted the invitation and has access.
- **Rejected** — roommate declined the invitation.
- **Revoked** — owner removed access after acceptance.

2.1.9.4. Decision Points

The flow includes key decision points:

- Roommate chooses to accept or reject the invitation.
- System verifies permissions before granting access.
- Owner may remove access at any time.

2.1.9.5. Access & Permissions

Once access is granted:

- The shared inventory appears in the roommate's **Shared Inventories** list.
- The roommate's actions depend on their assigned role:
  - Viewer → can view items.
  - Editor → can add or modify items.
  - Owner (optional) → full control.

2.1.9.6. Owner Management Actions

Inventory owners can:

- View who has access
- Change user roles (if supported)
- Revoke access
- Resend invitations

2.1.9.7. Purpose of the Flowchart

This flowchart helps:

- Designers understand the user experience
- Developers implement correct logic and states
- Team members maintain consistent sharing behavior
- Identify edge cases and permission handling

2.1.9.8. Invites Logic Definition

2.1.9.9. Objective

Define how invitation flows work between roommates so onboarding and access to shared resources is predictable and secure.

The purpose of this document is to define and describe:

```
Invitation lifecycle states:
```

- Sent
- Accepted
- Declined
- Expired
- Revoked
- Determine who is allowed to send invitations (owner only vs all members vs role-based).

  ```
  Specify validation rules:
  ```

- Duplicate invites
- Invalid emails
- Expiration timing

  ```
  Document expected behavior for:
  ```

- Invite acceptance

- Invite rejection
- Invite cancellation
- User joining via link vs direct invite

2.1.9.10. Core Concepts

Invitation

An **invitation** is a temporary request made by an Owner or authorized Member, that allows a user to join a household or shared inventory group.

Each invitation contains: - Sender - Recipient - Target household or group - Status - Expiration timestamp

Invitation Permissions

Permissions determine what user is allowed to send invitations.

Users with the appropriate permissions can include:

- Owner-Only
  - Only the household Owner may send invitations.
- Role-Based
  - Owners and designated Members may send invitations.

By default, invite permissions can only be made by Owners, unless Member permissions are expanded.

2.1.9.11. Invitation Lifecycle States

Sent

- Invitation has been created and delivered.
- Awaiting recipient response.
- Editable or revocable by sender.

Accepted

- Recipient has joined the household.
- Invitation is marked as accepted and becomes inactive.
- Permissions are assigned based on role configuraton.

Declined

- Recipient declines the invitation.
- Invitation is marked as declined and becomes inactive.
- Sender may resend a new invitation.

Expired

- Invitation exceeded its time limit.
- Cannot be accepted and is marked as inactive.
- Requires a new invitation to be sent.

Revoked

- Sender or Owner cancels the invitation before acceptance.
- Recipient loses the ability to join using that instance of the invitation.
- Marked as revoked and becomes inactive.

2.1.9.12. Validation Rules

Duplicate Invites

- Only allow invitations made to the same user for the same household to be editable, not duplicated.
- Allow resend only after expiration, decline, or revocation.

Expiration Timing

- Default expiration window: 72 hours (editable by inviter).
- System should automatically invalidate expired invites.
- Expired invites are not reusable.

2.1.9.13. Expected System Behaviors

Invite Acceptance

- User gains access immediately upon confirmation.
- Assigned default Member role unless specified otherwise.
- Notification sent to existing household members.

Invite Rejection

- Invitation marked Declined.
- No access granted.
- Sender notified of rejection.
- Household is notified of the declined invitation.

Invite Cancellation

- Sender or Owner may revoke before acceptance.
- Invitation becomes invalid instantly.
- Houselhold Members are notified of the cancellation.

- Recipient invite becomes unusable.

User Joining via Link vs Direct Invite

- Direct Invite
  - Recipient receives invite through application.
  - Invited user must have an account or create one to accept.
- Link Invite
  - Shareable link allows multiple recipients.
  - Requires validation step before access is granted.
  - Will require user to create an account or log in before joining.

2.1.9.14. Security Considerations

- Invitations must use secure tokens.
- Tokens should be single-use or time-bound.
- Server-side validation required for all joins.

2.1.9.15. Summary Rules

1. Invitations are temporary and must expire.
2. Only authorized roles may issue invites.
3. Active invites are editable as to avoid duplicate active invites.
4. Acceptance grants immediate access with default role.
5. Revoked or expired invites cannot be reused.
6. Server-side validation overrides client behavior.

2.1.9.16. Success Criteria

- Invitation lifecycle states clearly defined.
- Permission rules documented.
- Validation rules prevent misuse or duplication.
- Acceptance and rejection flows explicitly described.
- Understandable to both technical and non-technical team members.
- Ready to guide implementation of onboarding and access control.

2.1.10. Domain Invariants for Shared Inventory Ownership

2.1.10.1. Objective

This document defines the fundamental domain invariants governing inventories, members, items, and ownership relationships in a collaborative inventory system.

Domain invariants represent rules that must always hold true in the system. These rules establish the conceptual constraints necessary to support consistent ownership, sharing, and permission behavior.

The invariants defined here provide guidance for conceptual schema design and ownership rule definition.

2.1.10.2. Definition of Domain Invariant

A domain invariant is a rule that must always remain true for the system to be valid. Invariants describe required relationships between entities and define constraints on how resources can exist and interact.

Invariants are independent of implementation and represent structural truths of the domain.

2.1.10.3. Core Domain Invariants

Invariant 1: Every item belongs to exactly one inventory

Items exist only within a shared inventory context and cannot exist independently.

Implications:

- Item existence depends on Inventory.
- Ownership is scoped to an inventory context.
- Items cannot exist without a shared environment.

Conceptual relationship impact: Item → Inventory (required association)

Invariant 2: Ownership exists only within an inventory context

Ownership relationships are defined relative to a shared inventory and do not exist independently.

Implications:

- Ownership is contextual rather than global.
- Ownership must reference both an item and its inventory.

Conceptual relationship impact: Ownership depends on Item and Inventory.

Invariant 3: Only inventory members can own or manage items

Users may only own or manage items if they are members of the inventory that contains the item.

Implications:

- Ownership requires participation in the shared environment.
- Access control is grounded in membership.

Conceptual relationship impact: Ownership depends on Membership.

Invariant 4: Membership defines participation in shared resources

Users interact with items through their membership relationship with an inventory.

Implications:

- Users do not interact with items directly.
- Permissions derive from membership roles.

Conceptual relationship impact: User → Membership → Inventory → Item interaction chain.

Invariant 5: Every item must have a valid ownership state

Each item must have a defined responsibility structure, whether individual, collective, or hybrid.

Implications:

- Responsibility for items is never undefined.
- Ownership structure must be represented conceptually.

Conceptual relationship impact: Item must participate in an ownership relationship.

Invariant 6: Ownership cannot exist without an item

Ownership is a dependent relationship and cannot exist independently of a resource.

Implications:

- Prevents invalid ownership references.
- Ensures structural consistency of relationships.

Conceptual relationship impact: Ownership depends on Item existence.

Invariant 7: Membership requires a valid user and inventory

Membership is the relationship that connects individuals to shared inventories.

Implications:

- Shared context is required for participation.
- Ownership and permissions depend on membership.

Conceptual relationship impact: Membership connects User and Inventory.

2.1.10.4. Role of Invariants in Ownership Schema Design

These invariants establish the structural constraints that the ownership model must satisfy. They define:

- how items relate to inventories
- how users participate in ownership

- how responsibility is represented
- how permissions can be enforced

The invariants defined here provide a conceptual foundation for defining item ownership rules, inventory membership structure, and ownership schema representation.

2.2. Requirements

This section captures initial requirements at multiple levels (stories, features, epics, constraints).

2.2.1. User Stories

The following user stories describe the core user-visible behaviors of the Digital Home Inventory system for Milestone 1. Each story is written from the perspective of a persona and is supported by measurable acceptance criteria. The story identifiers use prefixes: US-INV (Core Inventory), US-ALT (Alerts & Restock), US-SB (Sharing & Budget), and US-RPT (Reports & Insights).

2.2.1.1. US-INV-01: Add items to inventory

As **Mateo** (busy solo student), I want to **quickly add items to my tracked inventory after shopping**, so that **I can reliably know what I have without relying on memory**.

Also affects: Sofia, Julian, Andrea **Priority:** High

Acceptance Criteria * The system provides a prominent “Add Item” entry point from the inventory view. * Adding an item requires only minimal fields (name + quantity/availability), with optional fields (category, location, ownership status, expiration date). * Creating an item takes low effort (goal: can be completed in under 1 minute for a small set of items, assuming typical mobile input). * After saving, the item appears in the inventory list without requiring a manual refresh. * The user can edit an item from the inventory list (e.g., correct name/variant), and the updated information is reflected consistently wherever it appears.

NOTE	Expiration dates are optional. If an expiration date is unknown or unreadable, the system treats expiration status as unknown and avoids false “expired” alerts.

2.2.1.2. US-INV-02: See what belongs to whom

As **Sofia** (budget-conscious roommate), I want to **see which items are shared and which belong to specific people**, so that **I can avoid conflict and protect my personal purchases**.

Also affects: Julian, Andrea **Priority:** High

Acceptance Criteria * Every item in a shared household includes an ownership indicator: personal (linked to a household member) or shared. * The item detail view shows ownership status and relevant metadata (purchase date/cost when recorded). * When adding/editing an item, the user can set ownership to personal or shared. * Changing ownership from personal shared updates the tracked record and becomes visible consistently to household members after synchronization. * Spending views (if enabled) separate personal spending from shared spending so users can understand personal vs household costs.

2.2.1.3. US-ALT-01: Receive low-stock warnings

As **Julian** (diet-constrained student), I want to **receive low-stock warnings for items I rely on**, so that **I can reorder in time and avoid unexpected shortages**.

Also affects: Mateo, Andrea **Priority:** High

Acceptance Criteria * The user can set a low-stock threshold for an item (with sensible defaults if no threshold is set). * When an item's tracked quantity crosses from NORMAL/IN_STOCK to LOW_STOCK (threshold boundary), the system flags the item as low-stock. * Low-stock alerts are generated primarily on state transitions (e.g., NORMAL → LOW_STOCK) and are deduplicated (re-running checks without state change does not spam alerts). * When multiple items are low, the system may group notifications or present a summarized view to avoid overwhelming users. * Low-stock status remains consistent across views (inventory list, restock list/planner, dashboard summaries where applicable).

2.2.1.4. US-SB-01: Split shared purchases with roommates

As **Andrea** (unofficial household manager), I want to **record shared purchases and split costs with roommates**, so that **reimbursements are clear and disputes are reduced**.

Also affects: Sofia, Mateo, Julian **Priority:** High

Acceptance Criteria * The user can record a purchase with total cost and date, and mark line items (or the purchase) as shared vs personal. * The system can compute a split for shared items among selected household members (default equal split; adjustable when needed). * Household members can view the breakdown of a shared purchase (what was bought, how the split was calculated). * Payments can be marked as paid/unpaid, and the system maintains a simple reimbursement status view. * Disputes can be flagged on a purchase line item (recording that a discussion is needed) without deleting transaction history.

2.2.1.5. US-RPT-01: See household dashboard with insights

As **Andrea** (unofficial household manager), I want to **open one screen and quickly see what needs attention**, so that **I can plan restocking without guessing or repeatedly messaging roommates**.

Also affects: Sofia, Mateo, Julian **Priority:** Medium

Acceptance Criteria * The dashboard presents a summary view including: number of low-stock items, number of expiring/expired items (when expiration dates exist), and current-period spending (if spending is enabled). * The user can filter the dashboard by location (e.g., kitchen/bathroom) and return to the unfiltered view. * Summary counts update after changes are recorded and synchronized (avoid stale information). * Low/expiring summaries include location context when available (e.g., "toilet paper low in bathroom"). * The dashboard does not require users to navigate multiple screens to find the "top-level" status of the household.

2.2.1.6. Features

Features describe user-visible capabilities of the system-to-be at a level above individual

requirements. The list below is intentionally concise for Milestone 1 and is grounded in the alert/grocery-list artifacts (auto-check output, trigger priorities, and the grocery list user flow).

ID	Feature	Primary value	Related stories / notes
F-INV	Inventory tracking (create/update items)	Enables a reliable tracked view of household items (quantity, location, ownership, optional expiration).	Supports US-INV-* (core inventory stories). Basis for alerts and grocery list generation.
F-ALERT	Alert generation from actionable state changes	Notifies household members when tracked conditions become actionable (e.g., LOW_STOCK, OUT_OF_STOCK, NEAR_EXPIRY/EXPIRED), with deduplication based on state transitions.	Supports US-ALT-* and the “alert trigger logic” notes. Delivery method can be in-app initially; push may be later.
F-AUTOCHECK	Automated daily check (AutoCheck) over inventory	Produces an “AutoCheck Output” (items + status results) used to drive alerts and grocery list updates without requiring the user to manually inspect everything.	Grounded in the AutoCheck/alert artifacts. Verification focuses on boundary cases (threshold equality, expiration date boundaries) and idempotency.
F-GL-AUTO	Auto-generated grocery list from AutoCheck Output	Keeps a grocery list updated based on priority triggers, reducing missed restocks and duplicate planning.	Consumes AutoCheck Output. Priority triggers (in order): (1) OUT_OF_STOCK, (2) EXPIRED, (3) LOW_STOCK, (4) FREQUENT & (LOW/OUT). Items already on the list are skipped. Actions are logged.
F-GL-USER	Grocery list review and editing flow	Users can review auto-added items, then remove/edit/add items manually; updated list is saved automatically.	Based on the grocery list user flow artifacts. Emphasizes user override and preventing clutter.

ID	Feature	Primary value	Related stories / notes
F-SPEND (optional scope if budgeting is in Milestone 1)	Spending capture and basic summaries	Allows household members to record spending and view simple period summaries (shared vs personal), supporting budget awareness.	Supports US-SB-* and US-RPT-* (basic reporting). Detailed finance flows can be deferred if needed.

NOTE	Feature F-GL-AUTO and F-GL-USER are based on the provided grocery list generator flow and user flow artifacts. Implementation details such as UI layout and backend scheduling are deferred to Section 2.3; here we capture only the user-visible capability and the trigger/priority semantics.

2.2.2. Domain Requirements

Requirements expressed in domain language (capabilities/outcomes), independent of UI/technology. Terminology follows the domain description: **Household**, **Household member**, **Location**, **Item type**, **Item instance**, and **Spending entry**.

2.2.2.1. Inventory integrity

- DR-1 (Barcode uniqueness): A single barcode shall map to exactly one **Item type**. The system shall not create a second item-type definition for an already-known barcode.
- DR-2 (Item instance validity): Every **Item instance** shall reference an existing **Item type** at the time it is recorded.
- DR-3 (Financial immutability): Removing an item from the tracked inventory (delete/archive/merge/split/adjust) shall not rewrite or delete associated **Spending entries**; financial history is immutable.

2.2.2.2. Shared-household rules

- DR-4 (Household administration): Each **Household** shall have exactly one household admin role at a time. Administrative responsibility shall be transferable before the current admin leaves.
- DR-5 (Membership independence): A system user may belong to multiple households, each with an independent membership role and membership state.
- DR-6 (Attribution retention): Spending entries shall remain attributed to the household member who made the purchase, even when inventory quantities are reorganized (merge/split) or when items are marked shared/personal.

2.2.2.3. Duplicate handling and intent preservation

- DR-7 (Short-window duplicates): If a duplicate item instance entry is detected within a short window (initially 30 seconds) with matching identity-relevant attributes, the tracked record shall aggregate quantity rather than create a separate entry (unless the user explicitly chooses

to keep them separate).

- DR-8 (Ambiguity guard): If item identity is ambiguous (near-duplicate name/brand/variant without authoritative identifier), the system shall not auto-merge without explicit confirmation.
- DR-9 (Auditability of corrections): Inventory corrections (merge/split/adjust) shall be auditable and reversible at the record level; the system shall preserve what changed, when, and why.

2.2.2.4. Reporting integrity (if reporting is in scope)

- DR-10 (Separation of concerns): Reports shall distinguish between inventory state (availability, locations, expiry) and spending history (amounts, dates, attribution); these shall not be conflated.
- DR-11 (Budget derivation): Budget/spending summaries shall be derived from spending entries within a defined budget period and shall not be altered by non-financial inventory corrections (e.g., discarding expired items).

2.2.2.5. Alerts and restock (Milestone 1 baseline)

- DR-12 (Threshold-driven low-stock): Low-stock shall be defined by an item-specific threshold (or default threshold when configured); crossing the threshold defines the low-stock condition.
- DR-13 (Expiration status): If an item instance has an expiration date, its expiration status shall be derivable (active/near-expiry/expired). If the expiration date is missing or unreadable, expiration status shall be treated as unknown (no false-expired condition).
- DR-14 (Deduplicated alerting): Alerts shall be deduplicated and triggered primarily on actionable state transitions (e.g., NORMAL → LOW_STOCK), so repeated checks without state change do not generate repeated alerts.

2.2.2.6. Prevention and data integrity

- DR-15 (Normalization before comparison): Product definitions (**Item type**) shall be normalized before duplicate comparison and before insert so that prevention logic is consistent and reproducible.
- DR-16 (Deterministic spend attachment): **Spending entries** shall be created and attached to product/inventory records in a deterministic way; duplicate resolution must be finalized before any spending entry is created.
- DR-17 (Single product per barcode at persistence): At persistence, one barcode shall map to exactly one **Item type**; duplicate prevention must enforce this before insert (consistent with DR-1).

2.2.2.7. Rationale / trace (Milestone 1)

- DR-1..DR-3 support reliability of tracked inventory and financial records (N2, and budgeting quality expectations if enabled).
- DR-4..DR-6 support shared-household accountability and coordination (N4, N5).
- DR-7..DR-9 support data quality and conflict reduction during entry/corrections (N1, N2, N3).
- DR-10..DR-11 support clear reporting and budget reasoning (if spending/reporting is in scope).

- DR-12..DR-14 support low-stock/expiry awareness with notification quality (N2, N3, and alert quality expectations).
- DR-15..DR-17 support duplicate prevention and data integrity (normalization, deterministic spend attachment, barcode uniqueness at persistence).

NOTE	Detailed policies and edge-case handling for sharing, permissions, and invitations are deferred for later milestones unless required by Milestone 1 deliverables. Keep the core domain requirements above stable and implementation-independent.

2.2.3. Interface Requirements

Interface requirements describe how phenomena and concepts from the domain are represented inside the system-to-be, how they are initialized, and how they are kept in sync when the real world changes.

2.2.3.1. IR-01: Item Initialization

When a household member acquires a new item in the real world, they must have a way to create a corresponding representation inside the system.

The system shall provide a means for household members to add new items to their tracked inventory, including name, quantity/availability, category, location, ownership status (personal/shared), and optional expiration date.

Questions this addresses: - How does an item first appear in the system? - What information is needed to represent what was acquired? - How does the system learn an item exists?

2.2.3.2. IR-02: Item Update on Consumption

When a household member consumes part of an item in the real world, the tracked representation must be updated to reflect new availability.

The system shall provide a means for household members to record consumption by decreasing recorded quantity/availability or marking an item as finished/out-of-stock.

Questions this addresses: - How does the system learn an item was partially/fully consumed? - How is tracked availability kept aligned with physical reality? - What happens when an item is finished?

2.2.3.3. IR-03: Ownership Specification

In shared households, members need to know whether an item is personal or shared.

The system shall provide a means to mark an item as personal or shared, and if personal, associate it with a household member.

Questions this addresses: - How does the system distinguish personal vs shared items? - Who can see/act on which items? - How are conflicts over personal items reduced?

2.2.3.4. IR-04: Low-Stock Threshold Definition

Households use different “low” definitions depending on the item.

The system shall provide a means to set low-stock thresholds for individual items (with sensible defaults when no threshold is set).

Questions this addresses: - How does the system know what “low” means for different items? - Can members customize thresholds to match their routines? - What happens when no threshold is provided?

2.2.3.5. IR-05: Expiration Date Entry

Some items have expiration dates that must be captured to support expiration awareness.

The system shall provide a means to enter and edit expiration dates for items that expire; items without expiration dates must be supported.

Questions this addresses: - How does the system learn when an item expires? - Is expiration required or optional? - How are items without expiration handled (no false confidence)?

2.2.3.6. IR-06: Purchase Logging

When a household member spends money on household items, that transaction must be representable in the system.

The system shall provide a means to log purchases, including items bought (or categories), total cost, date of purchase, and whether the purchase was personal or shared.

Questions this addresses: - How does spending information enter the system? - How can purchases be linked to items or categories? - How is shared vs personal spending distinguished?

2.2.3.7. IR-07: Cost Splitting

For shared purchases, costs may need to be divided among household members.

The system shall provide a means to split shared purchase costs among selected household members and track who owes what (according to household agreement).

Questions this addresses: - How are shared costs divided? - How does the system know who participates in a split? - How can disputes be reduced via clarity and traceability?

2.2.3.8. IR-08: Dashboard Metric Overview

Household members need a high-level overview without searching through details.

The system shall provide a dashboard view summarizing key metrics such as total items, low-stock items, expiring/expired items, and current-period spending.

Questions this addresses: - How does a member quickly assess what needs attention? - Which metrics matter most at a glance? - How are different tracked phenomena summarized consistently?

2.2.3.9. Appendix A: Edge-Case Interface Requirements (Deferred)

The following edge-case interface requirements (IR-EC-A through IR-EC-I) are recognized but deferred for detailed specification in later milestones. They are captured in this appendix, separate from core IRs, to avoid losing scope awareness and to support future traceability.

IR-EC-A: Near-Duplicate Variant Disambiguation

When a stakeholder adds an item whose name matches an existing product but whose size, variant, or brand differs, the system must present the conflicting attribute (size, variant, flavor) highlighted and require the stakeholder to choose: "This is a different variant (create new product)" or "Use existing product." Same-name/different-size entries shall be treated as separate product definitions by default and must not be auto-merged.

IR-EC-B: Stock Entry Separation by Expiration and Location

When a stakeholder attempts to merge two stock entries for the same product that have different expiration dates or different storage locations, the system must warn that merging will lose expiration/location tracking. The default action must be "Keep separate"; merging must require explicit confirmation.

IR-EC-C: Import Idempotency and Duplicate Spend Prevention

When a stakeholder uploads a CSV import file that was previously imported (detected by file hash), the system must default to "Review before apply" and present a summary of potential duplicate transactions. Per-row, if a transaction fingerprint matches a previously imported record, the system must skip spend event creation by default and require an explicit "Import anyway" override.

IR-EC-D: Inventory Correction Without Budget Impact

When a stakeholder deletes or merges an inventory entry that has associated spend events, the system must present two independent controls: "Remove from inventory" (default on) and "Also mark purchase as mistake — exclude from budget" (default off). Inventory correction must never delete spend events automatically. Both actions must be logged separately in the audit trail.

IR-EC-E: Refund and Return Flow

When a stakeholder returns a purchased item, the system must provide a refund flow that: decrements inventory quantity, creates a negative spend entry linked to the original where possible, and requires amount refunded, date, and an optional link to the original receipt or spend event.

IR-EC-F: Waste and Shrinkage Adjustment

When a stakeholder removes expired, lost, or consumed items from inventory outside of a normal purchase/return flow, the system must: allow inventory adjustment without altering any spend event, require an adjustment reason (Consumed / Lost / Shared use / Unknown), and offer an optional "Add to waste report" checkbox (default on for expired removals). Waste metrics must be reported separately from spending.

IR-EC-G: Shared Household Budget Attribution Prompt

When a stakeholder adds an item in a shared inventory, the system must prompt "Who bought this?" with the current user as default. If the stakeholder selects another household member, the system must require confirmation. Cross-user inventory merges are permitted for quantity counts, but each spend event must retain its original buyer attribution.

IR-EC-H: Undo for Auto-Merge Actions

Every auto-merge of inventory quantities (e.g., stock duplicate within 30-second window) must be reversible within 10 seconds via an undo control. Undo must revert quantity changes and record the undo in the audit log.

IR-EC-I: Audit Trail for Edge-Case Resolutions

Every edge-case resolution that modifies inventory state must generate a log entry containing at minimum: timestamp, user, household, source (manual / scan / import / adjustment / refund), action type, product reference, stock entry reference (if applicable), spend event reference (if applicable), reason (if user-selected), and a reference to the previous state. This log must be accessible to the household admin.

2.2.4. Machine Requirements

Machine requirements define measurable quality properties for Milestone 2 implementation. Since the prototype is under development, these requirements establish targets that will be verified upon completion. Where specific thresholds are not yet known, the requirement defines what will be measured and how success will be evaluated.

Each requirement includes traceability to relevant Domain Requirements (DR) where applicable.

2.2.4.1. MR-01: Availability

The system shall remain operational during supervised usability testing. No unplanned downtime requiring restart shall occur during testing sessions.

Trace: DR-12 (threshold-driven low-stock), DR-13 (expiration status)

Verification: Session logs; test coordinator observation.

2.2.4.2. MR-02: Response Time

The system shall respond quickly enough that household members can complete inventory tasks without waiting. During usability walkthroughs: - No participant shall abandon a task due to perceived slowness - All participants shall complete lookup tasks while simulating shopping (standing, holding phone) - Item creation shall be completable in under 1 minute (consistent with US-INV-01)

Response time measurements will be collected during walkthroughs to establish baseline for future milestones.

Trace: DR-7 (short-window duplicates), DR-12 (threshold-driven low-stock)

Verification: Usability walkthrough observations; timing recorded for each task.

2.2.4.3. MR-03: Offline Operation

The mobile client shall support viewing inventory while offline. Adding items and updating quantities while offline is deferred to post-Milestone 2.

Trace: DR-14 (deduplicated alerting)

Verification: Integration tests with connectivity simulation.

2.2.4.4. MR-04: Synchronization

When multiple household members use the same inventory, changes shall be propagated to all members in real time via Supabase Realtime. Changes shall appear without requiring manual refresh.

Trace: DR-4 (household administration), DR-5 (membership independence)

Verification: Automated tests verifying real-time delivery; manual tests with 2-3 concurrent users confirming changes appear instantly.

2.2.4.5. MR-05: Security and Privacy

Only household members shall have access to their respective household inventories. A user belonging to multiple households shall have access only to the household for which they are a member. Access revocation for a removed member shall take effect upon their next login attempt for that specific household.

Trace: DR-4 (household administration), DR-5 (membership independence), DR-6 (attribution retention)

Verification: Access control tests with test accounts spanning multiple households.

2.2.4.6. MR-06: Auditability

Quantity changes shall be logged with timestamp, user ID, item, and previous value.

Trace: DR-8 (ambiguity guard), DR-9 (auditability of corrections)

Verification: Schema validation; manual log inspection.

2.2.4.7. MR-07: Notification Quality

Repeated automated checks without state changes shall not generate duplicate alerts during testing.

Trace: DR-14 (deduplicated alerting)

Verification: Integration tests with repeated triggers.

2.2.4.8. MR-08: Barcode Scanning and Fallback

When a barcode is scanned, the system shall attempt to match it to an existing item type. If no match is found, the system shall allow manual entry without blocking the workflow. If the barcode is unreadable or scanning is unavailable, manual entry shall remain the primary fallback.

Trace: DR-1 (barcode uniqueness), DR-13 (expiration status fallback philosophy)

Verification: Manual tests with valid barcode, invalid barcode, and scanning unavailable scenarios.

2.2.4.9. Measurement Plan for Milestone 2 Completion

The following measurements will be collected during Milestone 2 validation activities to establish baseline for future milestones:

What to Measure	How	When
Response times	Record during usability walkthroughs	Post-prototype
Sync behavior	Verify real-time propagation via Supabase	During multi-user testing
Error occurrences	Log all exceptions during testing	Throughout testing
User perception	Participant feedback on speed/friction	During walkthroughs
Barcode scanning success rate	Track match vs manual entry ratio	During walkthroughs

These measurements will inform quantitative machine requirements for subsequent milestones.

2.2.4.10. Out of Scope for Milestone 2

Quantitative targets (specific response time percentiles, concurrent user capacity, uptime percentages) cannot be established without a prototype. These will be defined in Milestone 3 based on measurements collected during Milestone 2 validation.

2.3. Implementation

This section describes initial implementation plans and artifacts (architecture, design, diagrams, UI concepts). For Milestone 1, this is often a plan, not full implementation.

2.3.1. Software Architecture

High-level architecture: major components, responsibilities, and interactions.

2.3.1.1. Architecture overview

The team's solution follows a client backend architecture where a Flutter mobile application provides a user interface and local interaction flow, while the Supabase software provides the backend services which are required for authentication, data storage, file storage, and real-time updates. The Flutter app connects to Supabase over secure HTTPS using the Supabase client SDK, keeping the mobile client thin and focused on visual presentation, input validation, and offline friendly UX patterns.

The core data (items, households, members, and activity) is persisted in Postgres, which is managed by Supabase, and is protected using Row Level Security (RLS) policies so users can only access data belonging to their household(s). Media assets (e.g., receipts, item photos) are stored in Supabase Storage with access that is controlled by bucket policies and signed URLs if needed. When collaboration is required (e.g., shared lists or household changes), the app will subscribe to Supabase Realtime to receive updates without requiring a manual refresh.

2.3.1.2. Components

Component	Responsibility	Interfaces / dependencies
Flutter Mobile App	User interface, user interactions, basic client side validation, navigation, and state management. This displays inventory, shared supplies, and household context, initiates the CRUD operations and subscriptions.	Supabase Flutter SDK, device camera/filesystem for media capture, push notifications provider, local persistence.
Auth (Supabase Auth)	User registration/login, session management, password reset, and identity for authorization checks.	Supabase Auth, JWT sessions, email provider for verification/reset
API/Data Access Layer (Supabase PostgREST)	It exposes database tables/views as REST endpoints, executes the CRUD operations from the clien, enforces database constraints.	Postgres, PostgREST, RLS policies, Supabase client queries
Database (Supabase Postgres)	The primary persistent store for all structured application data (users, households, memberships, items, categories, transactions/activity logs).	Postgres, migrations/SQL, triggers/functions, RLS policies, indexes

Component	Responsibility	Interfaces / dependencies
Realtime (Supabase Realtime)	Publishes the changes to subscribed clients to support the collaboration (e.g., household inventory updates).	Supabase Realtime, database replication events, Flutter subscriptions
File Storage (Supabase Storage)	Stores and serves media assets such as: item photos, receipts, and attachments.	Storage buckets, signed URLs, bucket policies, Flutter upload/download via Supabase SDK
Security & Access Control (RLS + Policies)	Ensures the users can only read/write data allowed by their household membership and role.	Postgres RLS, policy definitions, JWT claims from Supabase Auth, role mapping (e.g., owner/member)

2.3.1.3. Data flow

- The user opens the app and signs using a Supabase Auth. Supabase returns a session (JWT) that the Flutter client then stores securely and attaches to subsequent requests.
- After the authentication, the app queries Supabase (PostgREST) for the user's households and memberships. RLS policies will restrict results to only the records the user is allowed to see.
- When the user creates/updates an inventory item, the Flutter app will send a create/update request to Supabase. Postgres constraints the validation of data and RLS verifies authorization before committing.
- If the user attaches an image (e.g., item photo or receipt), the app will upload the file to Supabase Storage, then store the resulting file path/URL reference in Postgres.
- The other members of the same household will receive updates through Supabase Realtime subscriptions (e.g., INSERT/UPDATE events) so their UI refreshes automatically.
- When the user views inventory details, the app will fetch item records from Postgres and, if needed, it will retrieve the media via Storage (public bucket) or via signed URLs (private bucket).

2.3.1.4. Risks and open questions

- Risk: An incorrect or overly permissive RLS policies could expose household data to unauthorized users.
- Risk: Realtime subscriptions may increase battery/network usage on mobile if it's not managed carefully (background behavior, reconnect storms).
- Risk: The media storage costs may grow quickly if images are not compressed, resized, or lifecycle-managed.
- Risk: Offline/poor connectivity may cause inconsistent user experience unless a local caching and conflict handling are designed.
- Open question: What roles are required (owner/admin/member/view-only) and what exact permissions should each role have?
- Open question: Should Storage buckets be private with signed URLs, or public with strict

naming + metadata controls?

- Open question: Do we need an audit/activity log table (who changed what/when) for accountability and debugging?
- Open question: What offline strategy do we want (read-only cache vs. offline edits with later sync/conflict resolution)?

2.3.2. Software Design

Design decisions at a more detailed level than architecture (modules, domain model, APIs, etc.).

2.3.2.1. Key design decisions

- **Decision 1: Use Supabase Realtime for live synchronization**

Supabase Realtime will be used to propagate inventory and budget updates instantly to connected clients. This decision was mainly made because the application requires the roommates to see changes like item additions, deletions, budget updates, etc. with minimal delay. Realtime's Postgres Changes and Broadcast features will reduce the need for manual polling and improve the overall user experience while integrating seamlessly with the existing Supabase stack.

- **Decision 2: Enforce access control using Supabase Row-Level Security (RLS)**

All data access permissions will be enforced at the database level using the RLS Policies. This ensures that users will only see items belonging to their household. Using RLS reduces backend complexity, prioritizing security and preventing any data leaks. Policies will rely on `auth.uid()` and `household_members` to determine authorization.

- **Decision 3: Use Edge Functions for complex business logic**

Supabase Edge Functions will handle operations that require transactional guarantees or cross-table validation such as budget enforcement, invite validation, or conflict resolution. RLS alone will be insufficient for these cases, so Edge Functions will provide serverside execution in a secure Deno Runtime to allow small operations using database transactions.

2.3.2.2. Domain model notes

The following mappings describe how domain concepts translate into system components:

- **User**
  - Source: Supabase Auth
  - Key field: `auth.uid()`
  - Related table: `users`
  - Purpose: ownership and identity
- **Household**
  - Table: `households`

    - Represents a shared living group
    - Links multiple users together
- **HouseholdMembership**
    - Table: `household_members`
    - Fields: `user_id`, `household_id`, `role`
    - Used by RLS policies to authorize access
    - Supports admin vs member roles
- **Home**
    - Table: `homes`
    - Represents the physical household environment where inventory is managed
    - Provides the structural context for rooms and stored items
- **Room**
    - Table: `rooms`
    - Represents a location within a home where inventory items may be stored
    - Associated with a Home through the `home_id` field
    - Examples include kitchen, pantry, and bathroom
- **Product**
    - Table: `products`
    - Represents a reusable product definition independent of a specific purchase
    - Used to normalize item metadata such as brand, size, and unit
- **Stock (Inventory Item Instance)**
    - Table: `stock`
    - Represents a purchased instance of a product tracked within the inventory
    - Includes quantity, status, purchase details, and expiration data
    - Real-time updates enabled via Supabase Realtime
    - Protected by RLS household policies
- **StockLocation**
    - Table: `stock_locations`
    - Represents the placement or movement of stock within rooms
    - Supports tracking of current location and location history
- **Inventory**
    - Table: `inventories`
    - Represents the collection of stock associated with a home
- **ShoppingList**
    - Table: `shopping_lists`

    - Represents planned purchases for restocking household inventory
    - Associated with a ShoppingListEntry through the `shopping_list_entries_id` field
- **ShoppingListEntry**
    - Table: `shopping_list_entries`
    - Represents individual items within a shopping list including quantity and priority
- **Category / Supplier**
    - Tables: `categories`, `suppliers`
    - Provide organizational and sourcing metadata for products
- **Budget / Expense**
    - Tables: `budgets`, `expenses`, `household_summary`
    - Validated through Edge Functions before writes
    - Updated automatically via database webhooks
- **InviteToken**
    - Table: `invite_tokens`
    - Generated via Edge Function
    - Short-lived signed token
    - Used by `/accept-invite` endpoint

Realtime Channels

- `inventory-updates`
    - Purpose: notify clients when items change
    - Mechanism: Supabase Realtime Postgres Changes or Broadcast
- `budget-updates`
    - Purpose: push household budget recalculations

Edge Function Endpoints

- **POST /validate-budget**
    - Input: `{ amount, household_id }`
    - **Behavior:**
        - Computes current monthly spending
        - Rejects if limit exceeded (403)
        - Otherwise approves write
    - Used before inserting expenses

- **POST /accept-invite**
  - Input: `{ token }`
  - **Behavior:**
    - Validates signed invite token
    - Adds user to `household_members`
    - Returns success/failure

RLS Policy Strategy

- SELECT policies filters by household membership
- INSERT policies verifies user belongs to household
- Admin-only actions use helper function:
  - `is_household_admin(h_id uuid)`
- Private items enforced via `owner_id = auth.uid()`

2.3.3. Database Schema

The database schema models users, homes, rooms, products, and stock (item instances). An earlier draft of the schema existed; it was revised in issue #122 (Lecture Topic Task: Inconsistencies with domain terminology and household item documentation).

The revision addressed a conflict between the Group-Owned Inventory model and the original stock representation: the original schema required each item to have exactly one owner. The relationship between Owner and Item was changed to many-to-many, and a new **Owns** table was added so an item can be owned by multiple owners while an owner can still own multiple items.

Here, users manage multiple homes that contain multiple rooms. Rooms can only belong to one home, and can contain multiple household items. Every item is contained in a room and references a product. Every item is also owned by a single owner. We want to track items belonging to owners, even if those owners are not users. An example of this would be a user attributing items to a roommate who is not themself a user.

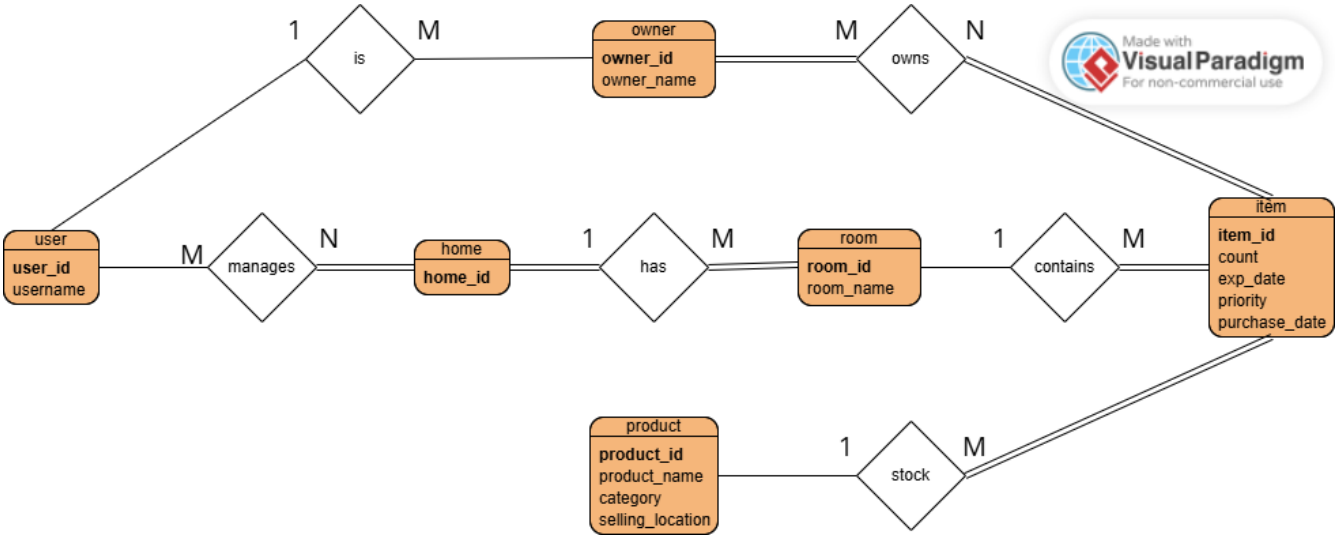

Figure 1: ER diagram for the project entities.

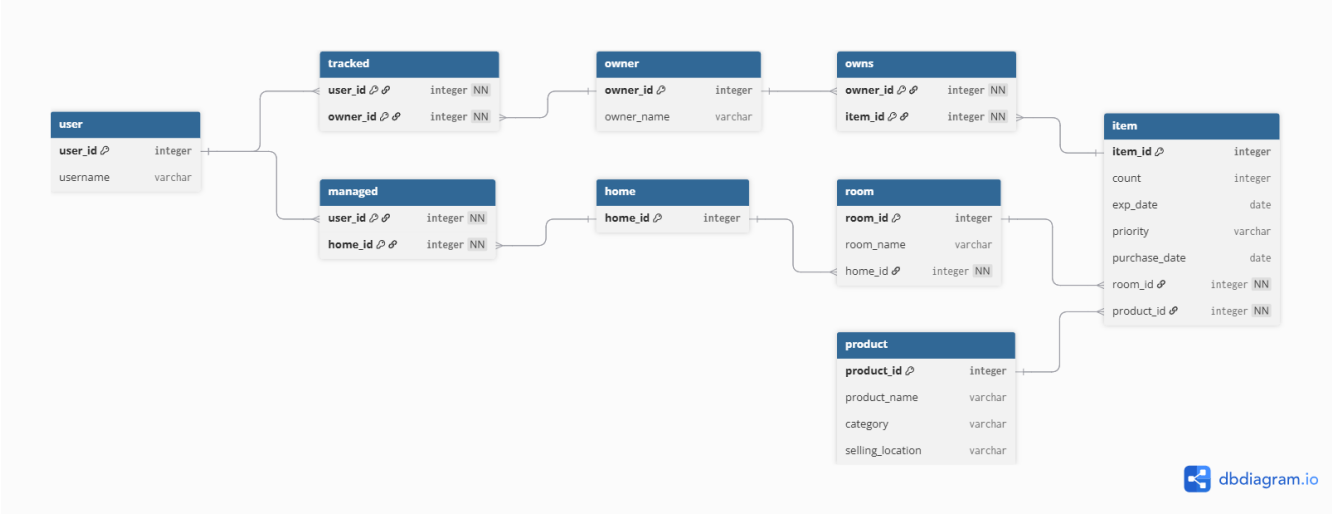

Figure 2: Table diagram for the backend.

2.3.4. Notifications Schema

This section describes the database schema for the Notifications & Alerts subsystem.

[notifications schema] | *notifications_schema.png*

Figure 1: Table diagram for the notifications system.

The notifications table stores in-app notifications for users.

Each notification: - Belongs to a user - May reference an inventory item - Has a type - Has a priority level - Has a status (pending, delivered, read, dismissed) - Includes timestamps for tracking

This design allows the system to store and manage notifications directly in the database.

2.3.5. Diagrams

Below is an inventory of diagrams included in this section:

Type	Filename	Purpose
Database Flow Diagram	stock_table_diagram.png	Shows backend database flows and system boundaries
Entity-Relationship Diagram (ERD)	stock_ERD.png	Shows finalized database schema and relationships

2.3.5.1. Database Flow Diagram

[stock table diagram] | *stock_table_diagram.png*

Shows the backend's database flows and how they interact. Useful for understanding system boundaries and responsibilities.

2.3.5.2. Entity-Relationship Diagram (ERD)

[stock ERD] | *stock_ERD.png*

Shows the finalized database schema including users, homes, rooms, items, products, and relationship tables. Useful for validating data structure and clarifying how features map to persistent storage.

2.3.6. Screen Designs

2.3.6.1. Key user flows

- Flow 1, Add Item to Household Inventory - User opens inventory, taps "Add Item", enters item details like name, quantity and location, and saves. With this the team ensures fast item capture and accurate household tracking.
- Flow 2, View & Respond to Alerts - User opens the Alerts & Notifications screen, reviews low stock or expiration alerts, and dismisses or fixes them. Purpose: help users maintain inventory awareness and prevent shortages.

2.3.6.2. Wireframes / mockups

- Alerts & Notifications Screen: [alerts wireframes]
- Budget Spending Page: [Budges spending wireframe]

NOTE Icon assets and small decorative images (e.g., error/empty icons, duplicate warning icons) are stored under the `Error-or-Empty-File-Icon/images` and `Duplicate-Warnings-Icon/images` folders. To avoid large icon pages in the main design narrative, these are referenced here as implementation artifacts. If the professor wants inline thumbnails, we can add them in a future revision.

2.3.6.3. Usability notes

- Prioritize one handed mobile use with clear, easy to press alert items.

- Alert states must be immediately understandable.
- Use simple, recognizable icons to reduce cognitive load.
- Limit visual clutter in data heavy screens like Budget Spending.
- Provide clear dismissal or action affordances for notifications.

3. Analytic Part

3.1. Concept Analysis

3.1.1. Domain Concept Analysis

This section derives domain concepts from the domain rough sketch (interviews, observations, edge situations). It documents how specific domain statements lead to abstractions (concepts), identifies potential term clashes, and defines relationships and invariants that hold in the domain independent of any system-to-be.

3.1.1.1. Starting point: raw statements held side-by-side

The following statements (from the rough sketch) are used as anchors for concept formation:

- Shared apartment: "If it is not on our fridge note, we assume we are out."
- Shared apartment: "Sometimes I check the pantry before leaving, but by the time I get to Walmart I forgot what we needed."
- Shared apartment: "We have rice in two places. One bag in the kitchen cabinet and another in a plastic bin in the hallway."
- Shared apartment: "Milk is usually bought twice in the same week at least once per month."
- Shared apartment: When asked how they know they are 'low': "when it feels light" or "when we open the backup." (and the backup opening is not recorded)
- Solo student: "If it takes more than a few seconds to log, I won't do it."
- Solo student: buys in bulk; yogurts sometimes expire; estimated 3–5 thrown away last semester.
- Household owner: child sometimes uses last item without saying; toilet paper runs out unexpectedly.
- Household owner: medicines kept separately; expiration dates important; thrown away expired medicine without realizing there was a second unopened box.
- Household owner: wants to remove access when someone moves out.
- Student athlete: consumption changes by context ("exam weeks"); buys based on memory of how fast it ran out last time.
- Brainstorming: people describe items as "full", "almost empty", "backup" (rarely with numbers).
- Edge situations: unreadable/missing expiration date; multiple expiration dates within a bulk package; backup stored in car trunk and forgotten.

These statements motivate the concepts below.

3.1.1.2. Concept formation: from statements to abstractions

We derive concepts by asking: (1) what phenomena exist in the domain, (2) what information households use to act, and (3) what changes over time.

Examples of abstraction:

- "If it is not on our fridge note, we assume we are out." → **Tracked representation** (note/list) vs **reality** (actual item availability), and **assumption under uncertainty**.
- "Rice in two places" → item storage **location**, and the possibility of multiple locations for the same item.
- "when it feels light" / "open the backup" → **low-stock judgment** that is qualitative and strategy-based, and **backup** as a household practice.
- "Milk bought twice in the same week" → **duplicate purchase** as a coordination failure, often arising from missing shared understanding.
- "3–5 yogurts thrown away last semester" / medicines expired with unopened box → **expiration** and **batches** (same item type can exist in multiple packages with different expiration dates).
- "remove access when someone moves out" → **membership** and **access to shared household information** (domain concept: who belongs to the household at a given time).

3.1.1.3. Core domain concepts (derived)

Concept	Definition (domain-level)
Household	A living unit (dorm, apartment, shared house) in which one or more people manage household essentials.
Household member	A person who currently belongs to a household and participates in consuming, replenishing, or coordinating around items.
Household item	A physical good kept for everyday use (e.g., milk, rice, toilet paper, soap, medicine).
Item type	The kind of item (e.g., "milk") independent of any specific package.
Item instance / package	A concrete package/unit of an item type (e.g., a specific milk carton, a specific box of medicine).
Inventory	The collection of household items that exist within a household at a given time.
Tracked inventory record	A representation of item availability used by household members (e.g., fridge note, mental model, list). This may diverge from the real inventory.
Location (storage place)	A physical place where items are stored (e.g., kitchen cabinet, hallway bin, bathroom closet, car trunk).
Availability	A household's current "how much is left" understanding, often qualitative ("full", "almost empty", "backup").
Quantity (optional formalization)	A more precise measure of availability (counts/units). The domain can operate without exact numbers.
Low-stock judgment	A decision that an item is running low, derived from strategies like "feels light", "opened backup", or a numeric threshold.

Concept	Definition (domain-level)
Backup supply	A household practice of keeping a spare instance/package to reduce the risk of running out.
Consumption	Use of items over time (may vary by context such as exams/guests).
Replenishment (restock)	Acquiring more of an item after consumption reduces availability.
Expiration date	A date after which an item instance/package is considered expired or undesirable.
Item batch (optional)	A set of item instances with the same expiration date (useful when bulk packages contain multiple expirations).
Coordination failure	A situation where household members act on inconsistent information (e.g., duplicate purchase, unexpected shortage).

3.1.1.4. Term clashes and clarifications

To prevent inconsistent usage across documentation, we clarify common ambiguous terms:

- **Inventory** (real-world items in the household) vs **Tracked inventory record** (note/list/memory). The rough sketch shows these diverge (e.g., “not on fridge note → assume out”).
- **Item type** (“milk”) vs **Item instance/package** (a specific carton). This matters for expiration and “second unopened box”.
- **Low-stock** is not inherently numeric; in the domain it is often qualitative (“feels light”, “open backup”).
- **Backup** is a domain concept (household practice), not an application feature.
- **Household membership** changes over time (“remove access when someone moves out”): membership is a domain phenomenon; access control is a system concern derived from it.

3.1.1.5. Relationships (domain model)

Concept A	Concept B	Relationship
Household	Household member	A household has one or more members; membership may change over time.
Household	Inventory	A household has an inventory (items that exist in the household).
Household item	Item type	Each household item corresponds to an item type.
Item type	Item instance/package	An item type may have multiple item instances/packages in the household at once (bulk buying, backup supply).

Concept A	Concept B	Relationship
Item instance/package	Location	Each item instance is stored in one location at a time; the same item type may be split across locations.
Item instance/package	Expiration date	Some item instances have an expiration date; others do not.
Household member	Consumption / replenishment	Members consume and replenish items; consumption rates vary by context (exam weeks, guests).
Tracked inventory record	Inventory (reality)	Tracked records approximate reality and may be incomplete, outdated, or wrong.
Household	Coordination failure	Coordination failures arise when members act on inconsistent or outdated tracked records.

3.1.1.6. Events, actions, and behaviors (domain-level)

- Events (instantaneous state changes):
  - An item becomes **out** (unexpected shortage).
  - An item becomes **low** (as judged by a strategy such as “backup opened”).
  - An item expires (or is discovered expired).
- Actions (intentional acts):
  - Check availability (pantry check, fridge note check).
  - Consume an item (use the last roll, eat snacks faster during exams).
  - Replenish an item (buy milk, buy bulk yogurt).
- Behaviors (multi-step ongoing processes):
  - Household maintenance cycle: consume → notice low/out → decide purchase → replenish → store → repeat.
  - Shared-household coordination: negotiate who buys what, or fail and duplicate-buy.

3.1.1.7. Invariants (domain rules)

These invariants describe stable properties of the domain:

- I1: A household has at least one member at any time it is considered active.
- I2: An item instance/package, if it exists, is stored in some location (even if the location is forgotten, e.g., car trunk).
- I3: Not all household judgments require numeric quantities; qualitative availability (“full”, “almost empty”, “backup”) is valid domain information.
- I4: If an item instance has an expiration date, it can become expired without being consumed, creating waste.
- I5: A household may possess multiple instances of the same item type simultaneously (bulk

buying, backups).

- I6: Tracked records can diverge from reality; decisions may be made based on assumptions under uncertainty ("not on note → assume out").

3.1.1.8. Edge cases and risks (from rough sketch)

- E1: Same item type stored in multiple locations (rice in cabinet + hallway bin) → risk of undercounting or duplicate purchase.
- E2: Backup supply opened but not recorded → low-stock event is not propagated to shared understanding.
- E3: Consumption rate varies by context (exam weeks, guests) → replenishment based on "last time" memory becomes unreliable.
- E4: Multiple expiration dates in a bulk package → motivates batch-level tracking if expiration awareness is important.
- E5: Expiration date unreadable/missing → household cannot reliably judge expiration status.
- E6: Membership change (move-out) → household needs to update who participates and who is informed.

3.1.2. Investigating Ownership Models for Collaborative Inventory Systems

3.1.2.1. Objective

This document analyzes conceptual ownership models applicable to shared-resource environments. The goal is to understand how ownership relationships may be structured in order to support sharing, permissions, and accountability within a collaborative inventory domain.

3.1.2.2. Definition of Ownership Model

An ownership model is defined as:

- the actor that holds primary responsibility for a resource
- the context in which ownership exists
- the authority and accountability implied by ownership

In collaborative inventory systems, ownership determines how items relate to users and shared inventories.

3.1.2.3. Ownership Model 1: User-Owned Resources

A resource is associated with a specific individual who holds primary responsibility and control independent of group context.

Relationship structure: Resource User

Characteristics

- Ownership is individual.

- Responsibility is clearly attributable.
- Sharing is an additional capability rather than inherent.

Advantages

- Clear accountability
- Simple responsibility tracking
- Intuitive model for personal assets

Limitations

- Weak support for inherently shared resources
- Requires additional mechanisms for collaboration
- Ownership may conflict with collective use cases

Implications for Shared Inventory Systems

This model is suitable when items represent individual contributions within a shared environment.

3.1.2.4. Ownership Model 2: Group-Owned Resources

A resource belongs to a collective entity such as an inventory or household. Users interact with the resource through membership rather than ownership.

Relationship structure: Resource Group

Characteristics

- Ownership is collective.
- Permissions derive from group roles.
- Resources exist only within a shared context.

Advantages

- Natural support for collaboration
- Consistent with communal resource management
- Simplified sharing semantics

Limitations

- No inherent individual accountability
- Responsibility attribution requires additional mechanisms
- Ownership transfer is not meaningful

Implications for Shared Inventory Systems

This model is appropriate for communal resources that are jointly maintained and used.

3.1.2.5. Ownership Model 3: Hybrid Ownership

A resource exists within a shared group context but is associated with a responsible member or subset of members.

Relationship structure: Resource Group and/or Resource Responsible Member (within group context)

Characteristics

- Combines shared context with individual responsibility
- Permissions may derive from both roles and ownership
- Supports accountability within collaboration

Advantages

- Supports both sharing and responsibility
- Flexible permission modeling
- Reflects real-world collaborative environments

Limitations

- Increased conceptual complexity
- Requires explicit ownership lifecycle definition
- More relationships must be modeled

Implications for Shared Inventory Systems

This model supports systems that require both collaborative access and responsibility attribution.

3.1.2.6. Comparative Summary

Model	Ownership Actor	Accountability	Sharing Support	Conceptual Complexity
User-Owned	Individual	Strong	Indirect	Low
Group-Owned	Collective	Weak	Native	Low
Hybrid	Group + Individual	Strong	Native	Medium

3.1.2.7. Relevance to Ownership Schema Design

The ownership model selected for a collaborative inventory system determines:

- how items relate to users and groups
- where ownership relationships exist conceptually
- how permissions and responsibility may be enforced

This analysis provides a conceptual foundation for defining item ownership rules and the inventory membership structure.

3.1.3. Evaluation of Group-Owned vs User-Owned Item Structures

3.1.3.1. Objective

This document evaluates conceptual ownership structures for items within a shared inventory system. The goal is to determine whether items should belong to individual users, the shared inventory, or follow a hybrid ownership structure.

This evaluation builds upon previously defined ownership models and domain invariants to identify the ownership structure that best supports collaborative use, accountability, and permission logic.

3.1.3.2. Evaluation Criteria

Ownership structures are evaluated based on their ability to:

- satisfy domain invariants governing items, membership, and inventories
- support collaborative interaction within a shared context
- provide clear responsibility attribution
- support enforceable permission logic
- represent domain relationships coherently

3.1.3.3. Ownership Structure 1: User-Owned Items

Conceptual Structure

Items belong to individual users and exist independently of group ownership. The shared inventory provides context for interaction but is not the owning entity.

Relationship structure: Item → User Item associated with Inventory context

Evaluation

Support for Domain Invariants	Collaboration Support	Accountability	Permission Logic	Conceptual Relationship Impact
• Partially satisfies item-in-inventory requirement. • Requires additional structure to maintain shared context. • Ownership is not inherently scoped to membership.	• Sharing must be explicitly added. • Not naturally aligned with collective resource management.	• Strong individual responsibility attribution.	• Ownership-based permissions are clear. • Group-based permissions become secondary.	User-centered ownership model with inventory as contextual container.

Assessment

User-owned structure provides strong accountability but weak alignment with shared-resource semantics.

3.1.3.4. Ownership Structure 2: Group-Owned Items

Conceptual Structure

Items belong to the shared inventory as a collective entity. Users interact with items through membership relationships.

Relationship structure: Inventory → Item User → Membership → Inventory

Evaluation

Support for Domain Invariants	Collaboration Support	Accountability	Permission Logic	Conceptual Relationship Impact
• Fully satisfies item-in-inventory requirement. • Ownership scoped to shared context. • Strong alignment with membership-based interaction.	• Native support for shared resource interaction.	• Responsibility attribution is not inherent.	• Permissions derive from membership roles.	Inventory-centered ownership model with membership-mediated interaction.

Assessment

Group-owned structure strongly supports collaboration but lacks inherent responsibility attribution.

3.1.3.5. Ownership Structure 3: Hybrid Ownership

Conceptual Structure

Items exist within a shared inventory context but are associated with a responsible member within that context.

Relationship structure: Inventory → Item Membership → Responsibility for Item

Evaluation

Support for Domain Invariants	Collaboration Support	Accountability	Permission Logic	Conceptual Relationship Impact
• Fully satisfies item-in-inventory requirement. • Ownership scoped to membership context. • Consistent with shared participation model.	• Native support for shared interaction.	• Clear responsibility attribution within shared context.	• Supports combination of role-based and ownership-based permissions.	Shared context with scoped individual responsibility.

Assessment

Hybrid ownership balances collaboration and accountability while maintaining alignment with domain invariants.

3.1.3.6. Comparative Summary

Structure	Collaboration	Accountability	Invariant Alignment	Conceptual Fit
User-Owned	Indirect	Strong	Partial	Moderate
Group-Owned	Native	Weak	Strong	Strong
Hybrid	Native	Strong	Strong	Strong

3.1.3.7. Recommendation

Based on domain invariants and shared-resource requirements, the hybrid ownership structure provides the most complete conceptual model. It preserves shared context while enabling responsibility attribution and flexible permission logic.

This structure supports collaborative interaction, aligns ownership with membership, and provides a coherent foundation for defining item ownership rules and schema relationships.

3.1.4. Ownership Implications for Permissions and Responsibility Attribution

3.1.4.1. Objective

This document analyzes how item ownership influences permissions and responsibility attribution within a shared inventory system. The goal is to ensure that the selected ownership structure supports enforceable control, accountability, and consistent system behavior.

This analysis builds upon previously defined ownership models, domain invariants, and the selected hybrid ownership structure.

3.1.4.2. Ownership as a Behavioral Concept

Ownership is not only a structural relationship but also a behavioral one. It defines authority over resources and identifies responsibility within a shared context.

In a collaborative inventory domain, ownership must support:

- permission determination
- responsibility attribution
- consistent interaction within shared environments

3.1.4.3. Relationship Between Ownership and Membership

Ownership exists within an inventory context and is scoped by membership. Users interact with items through their participation in the shared inventory.

Conceptual interaction structure:

User → Membership → Inventory → Item Membership → Responsibility for Item

Implications:

- ownership requires membership
- permissions are constrained by shared context
- responsibility is attributed within group participation

3.1.4.4. Ownership and Permission Control

Ownership contributes to permission determination by identifying the participant with primary authority over an item.

Permission sources include:

- membership roles defining baseline capabilities
- ownership responsibility defining elevated authority

Conceptual permission structure:

Responsible Member	Inventory Member	Inventory Administrators
• may manage item state and properties • holds primary decision authority	• interact according to membership role • do not possess inherent ownership authority	• may manage ownership assignments • may override ownership constraints

Implications:

- ownership refines authority within shared access
- permission logic depends on both membership and ownership relationships
- authority is scoped to shared context

3.1.4.5. Ownership and Responsibility Attribution

Ownership provides a mechanism for associating responsibility with shared resources.

Conceptual responsibility model:

- items exist within shared context
- ownership identifies responsible participant
- actions involving items can be attributed through ownership

Implications:

- accountability is preserved in collaborative environments
- responsibility is never undefined
- ownership supports attribution of actions or resource-related outcomes

3.1.4.6. Conceptual Requirements for Ownership Structure

To support permission control and responsibility attribution, the ownership model must ensure:

- ownership exists within an inventory context
- only members may hold ownership responsibility
- items always have a defined responsibility structure
- permissions can be derived from ownership and membership relationships
- responsibility attribution is structurally supported

3.1.4.7. Implications for Conceptual Schema Design

This analysis establishes behavioral constraints that the conceptual schema must support:

- items must reference an inventory context
- membership relationships must connect users to shared resources
- ownership responsibility must be representable within membership scope
- permission and responsibility relationships must be derivable from conceptual structure

These findings validate the hybrid ownership model as a coherent foundation for defining item ownership rules and shared inventory interaction behavior.

3.1.4.8. Domain Attributes Classification for Core Inventory

This section applies the lecture topic Domain Attributes to the Core Inventory subsystem by classifying key domain entities and behaviors according to their attributes (Static, Dynamic,

Dynamic Inert, Dynamic Active, Discrete).

1. Product

Represents an item definition (e.g., "Clorox Bleach 64oz Original").

Classification: *Static *Discrete

A product's definition does not change over time once created. Its properties (name, barcode, brand, size) remain constant and if we use the example in the beginning a "Clorox Bleach 64oz Original" will remain the same regardless of the passage of time, thus it is classified as Static. It is also Discrete because each product is its own distinct entity in the system.

2. Item Stock Entry

Represents a specific purchase of a product, it includes information such as date of purchase, location of storage and date of expiration if applicable.

Classification: *Dynamic Inert *Discrete

An item changes only due to external actions such as user updates of the product, consumption, or location change. Since it is changed via external stimuli from the user it is considered Dynamic Inert. Each item has a separate record in the system, making it Discrete.

3. Home

Represents a household container that groups rooms and inventory items.

Classification: *Static *Discrete

A home is a structural entity within the system and does not change state over time since it serves as a container for other entities. Each home is unique making it Discrete.

4. Room

Represents a physical location within a home (e.g.,bedroom, pantry, fridge).

Classification: *Static *Discrete

Rooms are structural subdivisions of a home and do not change autonomously meaning that they are Static entities. Each room is uniquely represented, so it is Discrete.

5. Owner

Represents the person associated with a specific item.

Classification: *Static *Discrete

An owner's identity does not change over time, while item associations may change, the owner entity itself remains stable so it is Static. Each owner is their own distinct entity, making it Discrete.

6. Duplicate Detection Logic

System logic that determines whether a new entry is a duplicate at either the product or the stock level.

Classification: *Dynamic Active

Duplicate detection executes a rule-based behavior when users add or scan items. It responds to system events and applies programmed rule conditions. Therefore, the Duplicate Detection Logic is classified as Dynamic Active since it can change via internally programmed rules and various conditions.

Evidence Concept Analysis

This subsection documents how raw observations from the domain rough sketch were transformed into structured domain concepts. The goal is to ensure traceability between the initial unstructured notes and the final abstractions used in the domain model.

Rough Sketch vs Concept Formation

Rough Sketch Statement	Derived Concept
"All participants said they have at some point forgotten what they already had at home." (Brainstorming session)	Household Item
"Milk is usually bought twice in the same week at least once per month." (Shared apartment interview)	Duplicate purchase / Coordination failure
"Yogurts sometimes expire; estimated 3–5 thrown away last semester." (Solo student interview)	Expiration Date
"when we open the backup." / "We have rice in two places." (Shared apartment interview)	Backup supply (multiple instances of a Household Item)
"toilet paper runs out unexpectedly when guests visit." (Household owner interview)	Availability (qualitative state: full, almost empty, backup) / Low-stock judgment

By analyzing these statements side-by-side:

- Different expressions referring to "groceries", "items", and "products" are unified under the concept of **Household Item**.
- Repeated mentions of duplicates and shortages indicate coordination issues within shared households.
- Time-based concerns introduce the concept of **Expiration Date** tied to specific item instances.

Concept Formation Decisions

The following abstractions were introduced by grouping similar ideas:

- "Groceries", "items", "products" - **Household Item**
- Multiple physical units - **Item Instance**
- "Extra one" - **Backup supply**
- "Expires" - **Expiration Date**
- "Running out" - **Availability** (with qualitative judgments: full, almost empty, backup)

These groupings allow consistent reasoning across different user statements and scenarios.

Identifying Concept Relationships

From the analysis:

- A **Household Item** may have multiple **Item Instances**
- Each **Item Instance** may have an **Expiration Date**
- A **Household Item** can be associated with an **Availability** state (qualitative assessment: full, almost empty, backup)
- Household members interact with shared items in a common environment

Terminology Alignment

Potential inconsistencies were identified and resolved:

- "Item", "product", and "groceries" - standardized as **Household Item**
- "Running out" vs "not available" - unified under **Availability**
- "Extra" vs "backup" - unified as **Backup supply**

Domain Scope Note

This analysis focuses strictly on domain concepts and avoids system-specific terminology. Instead of referring to "users", domain roles such as household members, roommates, or students are implied.

This section highlights a representative subset of concept formation decisions. The parent concept analysis derives approximately 15 distinct concepts; this section traces 5 foundational concepts to demonstrate the methodology of mapping raw observations to domain abstractions.

3.1.4.9. Trace to earlier documentation

This domain concept model is grounded in the domain rough sketch’s concrete examples of informal tracking, multiple storage locations, qualitative low-stock judgments, bulk buying, expiration waste, and shared-household coordination failures. It supports the needs described in the informative part (minimizing effort while maintaining reliable availability understanding, and improving shared-household coordination) without committing to specific implementation decisions.

3.1.5. Requirements Concept Analysis

This section analyzes requirement-related concepts (actors, goals, capabilities, and requirement-level vocabulary) and maps them to the domain concepts established in the Domain Concept Analysis. It is grounded in (a) the Domain Rough Sketch (interviews/observations) and (b) the project's internal Milestone 1 notes located in the docs directory (automation checks, low-stock strategies, expiration alert definitions, notification evaluation, grocery-list auto-add logic, transaction tracking, and schema notes). The goal is to keep later requirements (Section 2.2) precise, consistent, and traceable.

3.1.5.1. Starting point: requirement drivers anchored in the rough sketch

The rough sketch motivates these requirement drivers:

- **Low friction capture:** "If it takes more than a few seconds to log, I won't do it."
- **Shared coordination failures:** milk bought twice; no formal responsibility; reliance on fridge note + chat.
- **Multiple storage locations:** rice stored in two places.
- **Qualitative stock judgments:** "full", "almost empty", "backup"; low = "feels light" / "open the backup" (and backup-opening often not recorded).
- **Expiration and waste:** food/medicine expires unnoticed; unopened duplicates exist; unreadable/missing expiration; multiple expirations in bulk packs.
- **Variable consumption:** exam weeks/guests change usage rates.
- **Membership change:** desire to remove access when someone moves out.

These drivers shape the requirement-level concepts and tradeoffs below.

3.1.5.2. Actors and goals

Actor (domain role)	Primary goals (outcomes)
Student living independently (solo)	Maintain a reliable understanding of what is available with minimal effort; avoid duplicate purchases; avoid unexpected shortages; reduce waste from expired items.
Student living with roommates/cohabitants	Maintain a shared understanding of availability; coordinate replenishment to avoid duplicates/missed purchases; distinguish shared vs personal items without conflict.
Household member with budget-awareness focus (persona trait)	Understand spending patterns for essentials; avoid overspending; plan purchases using summaries rather than memory.
Household organizer (persona trait)	Reduce confusion about where items are stored and how they are grouped; keep organization stable enough that others can follow it.

Actor (domain role)	Primary goals (outcomes)
Operations/monitoring stakeholder (class-topic persona)	Ensure automated checks and alert generation are reliable, observable, and do not produce duplicates or noisy output.

3.1.5.3. Requirement-level concepts (definitions and clarifications)

The following concepts are used to express requirements without committing to a specific implementation:

Concept	Meaning in requirements
Fast capture	A tracking/update interaction that fits within a few seconds and does not require perfect detail at entry time.
Tracked record vs reality	The system-to-be represents availability, but real household inventory changes continuously; divergence is expected and must be managed.
Shared visibility	Household members can converge on the same understanding of availability and recent changes. “Real-time” is treated as a quality expectation, not a domain concept.
Actionable states	States that drive decisions: e.g., NORMAL, LOW_STOCK, OUT_OF_STOCK, NEAR_EXPIRY, EXPIRED.
State transition	A change from one actionable state to another. Transitions are a primary trigger for alerts to reduce repeated notifications.
Low-stock strategy	How “low” is determined (fixed threshold, percentage-based, reorder point). Initial requirements may accept a fixed per-item threshold.
Near-expiry window	Definition of “expiring soon” (within N days), including any timing/grace policy to avoid inconvenient alert timing.
Scheduled sweep (automation check)	A periodic background check that evaluates low-stock and expiration conditions without requiring the user to open the system.
Notification record/event	A persisted representation of an alert (at least for in-app review), including type, time, and read/unread status.
Delivery approach	In-app notification list vs device-level push. Choice affects complexity, reliability, and verification approach.
Deduplication / anti-spam	Rules preventing repeated alerts or repeated list additions when nothing meaningful changed.
Auto-add to grocery list	Rule-based conversion of actionable states into grocery list entries, with guardrails against clutter.
Transaction history	An append-only log of quantity changes with reason and timestamp, enabling traceability of how current quantity was reached.

3.1.5.4. Capability map (actors → capabilities → domain concepts)

Actor	Capability needed	Domain concepts involved
Solo student	Capture/update items quickly (create, adjust availability, mark used/restocked)	Tracked inventory record, Item type/instance, Availability/Quantity, Consumption, Replenishment, Location, Category
Solo student	Check availability before shopping (search/filter; see current status)	Tracked inventory record, Availability/Quantity, Location, Category, Actionable states
Solo student	Reduce expiration waste (see expiring/expired items; decide actions)	Item instance/package, Expiration date, (optional) Batch, Expiration states
Roommate/cohabitant	Maintain shared visibility (see the same inventory state and recent changes)	Household, Household member, Tracked records, Coordination failure
Roommate/cohabitant	Coordinate replenishment intent/responsibility to avoid duplicates	Low-stock judgment/Threshold, Replenishment intent/responsibility, Grocery list entry (if used)
Budget-focused member	Record/review spending summaries for essentials	Spending entry (optional), Category, Household
Operations/monitoring stakeholder	Reliable automated checks with observability and deduplication	Scheduled sweep, Run results/logs, State transitions, Notification records/events

3.1.5.5. Requirement groupings (what must be specified later)

These groupings organize later requirements (Section 2.2) and help keep coverage balanced.

RG1 — Inventory maintenance (core) * Add items with minimal burden. * Update availability through common actions (used/consumed, restocked, manual adjustment). * Organize items by category and optionally by location. * Support multiple storage locations for the same item type (either separate records or multi-location representation).

RG2 — Shared-household coordination * Shared visibility of inventory and recent changes. * Distinguish shared vs personal items. * Represent replenishment intent/responsibility to reduce duplicate buying and missed purchases.

RG3 — Low-stock and out-of-stock awareness * Low-stock determination based on a simple, explainable strategy initially (fixed per-item threshold acceptable). * Out-of-stock as a distinct actionable state. * Alert generation tied to state transitions and deduplication rules.

RG4 — Expiration awareness * Expiration applies to some items; others have no expiration date. * “Expiring soon” concept defined via near-expiry window (+ optional grace policy). * Handle missing/unreadable expiration dates without creating false confidence.

RG5 — Automation layer (scheduled sweep) * Periodic sweep evaluates low-stock and expiration conditions. * Sweep produces persisted state flags and generates notification records/events for new actionable states. * Sweep is observable (run summaries/logs) and idempotent (safe retries without duplication).

RG6 — Grocery list auto-add (rule-based) * Auto-add triggers based on actionable states: OUT_OF_STOCK, EXPIRED, LOW_STOCK (optionally with "frequent pattern" gating). * Auto-add must not duplicate items already on the list. * Trigger priority and reason must be explainable ("why was this added?").

RG7 — Transaction traceability * Every availability change is recorded as a transaction with reason and timestamp. * Transaction history is append-only (read-only after creation). * Current quantity remains consistent with applied transactions.

3.1.5.6. Evidence for requirement concepts (artifact anchors)

The requirement-level concepts and decision-rule vocabulary above are anchored in project notes located in the docs directory (filenames shown below). These artifacts serve as the retraceable basis for later requirements and tests.

- Scheduled sweep / automation checks: `automation-check-system.adoc`
- Expiration alert definitions and near-expiry window: `definition-expiration-alert-system.adoc`
- Notification fatigue concerns and evaluation criteria: `evaluationOfNotification.adoc`
- Grocery list auto-add priority and de-duplication logic: `grocerylist_autoadd_decisionlogic.adoc`
- Transaction history and append-only tracking of quantity changes: `inventory-transaction-tracking.adoc`
- Low-stock determination alternatives: `low-stock-research-strategies.adoc`
- Inventory/stock-change schema notes used to align terminology: `schema-for-inventory-&-stock-changes.adoc` (and related notification schema notes where applicable)

Evidence Concept Analysis

This subsection documents how requirement-level concepts were derived from raw observations in the rough sketch and supporting research artifacts. It ensures traceability between real-world problems and the 13 requirement-level concepts defined in the parent section.

Rough Sketch Drivers to Requirement Concepts

Rough Sketch Driver	Rough Sketch Evidence	Requirement-Level Concept(s)
Low friction capture	"If it takes more than a few seconds to log, I won't do it"	Fast capture
Shared coordination failures	"Sometimes we buy the same item twice" / No formal responsibility / Reliance on fridge note + chat	Shared visibility, State transition, Deduplication / anti-spam, Auto-add to grocery list

Rough Sketch Driver	Rough Sketch Evidence	Requirement-Level Concept(s)
Multiple storage locations	"We have rice in two places." (Shared apartment interview)	Tracked record vs reality (divergence between stored locations and system record)
Qualitative stock judgments	"full", "almost empty", "backup"; "feels light" / "open the backup" (often not recorded)	Actionable states, Low-stock strategy, Availability representation
Expiration and waste	"I don't notice when things expire" / Yogurts expire; 3-5 thrown away per semester	Near-expiry window, Notification record/event, Delivery approach, State transition, Scheduled sweep
Variable consumption	Exam weeks / guest visits change usage rates / "toilet paper runs out unexpectedly when guests visit"	Actionable states (demand-dependent), Low-stock strategy (flexibility)
Membership change	Desire to remove access when someone moves out	Shared visibility (access control implications), Transaction history (audit trail)
Tracking reliability	"I check the fridge note before buying" — system used as reference point	Tracked record vs reality, Transaction history, Notification record/event

Concept Formation from Evidence and Artifacts

The 13 requirement-level concepts were formed by analyzing rough sketch statements, supporting research documents, and actor capabilities. The following table traces the originating evidence for each concept:

Requirement Concept	Evidence Source	Key Implication
Fast capture	"If it takes more than a few seconds to log, I won't do it"	Interaction design must minimize burden (few fields, simple confirmation)
Tracked record vs reality	"I check the fridge note before buying"; "We have rice in two places." / household inventory changes continuously	System represents availability at a point in time; divergence is inevitable and must be managed
Shared visibility	"Sometimes we buy the same item twice" / No formal responsibility / Reliance on fridge note + chat	Multiple household members must converge on the same understanding without external coordination tools

Requirement Concept	Evidence Source	Key Implication
Actionable states	Exam weeks/guest visits cause demand spikes; "toilet paper runs out unexpectedly"	States must represent decision triggers, not just raw facts (NORMAL, LOW_STOCK, OUT_OF_STOCK, NEAR_EXPIRY, EXPIRED)
State transition	Repeated mentions of duplicates and alerts / Need to avoid notification fatigue	Alerts should trigger on **changes** (transitions), not on static conditions remaining unchanged
Low-stock strategy	Variable consumption / "runs out unexpectedly" / research in `low-stock-research-strategies.adoc`	Fixed per-item threshold selected for balance between simplicity and flexibility (recommended in artifact)
Near-expiry window	"I don't notice when things expire" / research in `definition-expiration-alert-system.adoc`	Definition of "expiring soon" (N days) required to alert before waste occurs; grace periods needed
Scheduled sweep (automation check)	Expiration and stock concerns; need for monitoring beyond user interactions / `automation-check-system.adoc`	Daily background process evaluates low-stock and expiration conditions; generates state-transition notifications
Notification record/event	Alerts required for expiration and low-stock / `evaluationOfNotification.adoc`	Persisted representation of alerts (at least read/unread status in-app) needed for review and deduplication
Delivery approach	"I don't notice when things expire" (delayed awareness) / research in `evaluationOfNotification.adoc`	In-app notification records selected for Milestone 1 (simpler, testable); push notifications remain optional
Deduplication / anti-spam	"Sometimes we buy the same item twice" (coordination failures) / need to avoid notification fatigue	Rules required to prevent repeated alerts when state unchanged; repeated list additions when already recorded
Auto-add to grocery list	Duplicate purchases + low-stock concerns; coordination through fridge note / `grocerylist_autoadd_decisionlogic.adoc`	Rule-based triggers (OUT_OF_STOCK, EXPIRED, LOW_STOCK) convert actionable states into grocery list entries

Requirement Concept	Evidence Source	Key Implication
Transaction history	Tracking reliability / "I check the fridge note before buying" / `inventory-transaction-tracking.adoc`	Append-only log of quantity changes with reason and timestamp enables traceability and audit; current quantity derives from transactions

Requirement Formation Decisions

The following requirement concepts were selected because they directly address domain problems identified in the rough sketch and supported by research artifacts:

- **Fast capture** — directly addresses "won't log if > few seconds" friction
- **Tracked record vs reality** — acknowledges that households are dynamic; system is a representation, not ground truth
- **Shared visibility** — necessary to reduce duplicate purchases and informal fridge-note coordination
- **Actionable states** — enables decision-making without requiring users to interpret raw facts
- **State transition** — critical to alert design; prevents notification fatigue
- **Low-stock strategy** — balances simplicity (fixed threshold) with flexibility (per-item configuration)
- **Near-expiry window & scheduled sweep** — automate detection of waste risks
- **Notification record/event & delivery approach** — provide a way to notify without requiring push infrastructure
- **Deduplication / anti-spam** — protects against alert fatigue and repeated grocery list additions
- **Auto-add to grocery list** — extends alerts into actionable purchasing recommendations
- **Transaction history** — provides evidence trail for shared accountability and consumption analysis

Alignment with Domain Concepts

Requirement-level concepts align with domain concepts as follows:

- Fast capture - operations on **Household Item**
- Tracked record vs reality - gap between **real-world Household Item** and **Inventory system representation**
- Shared visibility - **Household** (shared context) and interaction between **Household members**
- Actionable states - **Availability** (qualitative state: full, almost empty, backup) and derived judgment states
- State transition - triggering mechanism for **notifications**
- Low-stock strategy - implementation of **low-stock judgment** concept

- Near-expiry window - definition of **expiration-time concern**
- Scheduled sweep - systematic evaluation of **Item Instance expiration** and **Availability**
- Notification record/event - recording mechanism for system responses
- Delivery approach - infrastructure choice for user communication
- Deduplication / anti-spam - rules governing **notification** production
- Auto-add to grocery list - connection between **Household Item state** and **purchasing intent**
- Transaction history - evidence trail for **Household Item** state changes

This ensures consistency between domain analysis (Section 3.1) and requirements.

Scope Note

This evidence section traces all 13 requirement-level concepts to their originating rough sketch statements and supporting research artifacts. Concepts include both domain-grounded concerns (fast capture, shared visibility, actionable states) and system-design concerns (scheduled sweep, notification delivery, deduplication). While the latter may involve system behavior, they remain justified by domain drivers and supported by architectural research documented in the supporting artifacts referenced above.

3.1.5.7. Known decision logic (expressed as requirement-level rules)

The following rules are expressed at the requirements level (not as implementation details). They should later become explicit requirements with fit criteria.

Scheduled sweep evaluation rules * Low stock: LOW_STOCK becomes true when availability indicates quantity ⇐ low_stock_threshold (when a threshold exists). * Expired: EXPIRED becomes true when today >= expiration_date (when an expiration date exists). * Near-expiry (optional): NEAR_EXPIRY becomes true when expiration_date - today ⇐ N days (when an expiration date exists). * Alerts should be produced on **new** actionable states (state transitions) to avoid repeated notifications for unchanged conditions.

Grocery list auto-add decision logic (priority + deduplication) * If OUT_OF_STOCK and not already on list → auto-add (reason OUT_OF_STOCK). * Else if EXPIRED and not already on list → auto-add (reason EXPIRED). * Else if LOW_STOCK and not already on list → auto-add (reason LOW_STOCK). * “Frequent pattern” signals (if used) must be gated behind low/out conditions and must not override stock reality.

Transaction tracking rule * Any action that changes quantity is recorded as a transaction (restock, used/consumed, manual adjustment), with reason and timestamp.

3.1.5.8. Notification delivery: requirement implications of approach options

Two delivery approaches have different requirement implications:

- In-app notification records:
  - Requires storing notification records and supporting read/unread state.
  - Easier to validate/verify early because behavior is visible and testable within the system.

- Push notifications:
  - Adds device registration/token lifecycle, retries, platform-specific delivery behavior, and higher verification burden.

For Milestone 1 scope, in-app notification records are an acceptable baseline; push delivery remains an optional extension.

3.1.5.9. Conflicts and tradeoffs (requirement-level)

- T1: **Speed vs accuracy** — demanding full detail slows capture; minimal capture risks missing expiration/location details.
- T2: **Flexibility vs consistency** — free-form categories/labels match households but increase terminology drift, especially with roommates.
- T3: **Shared visibility vs privacy** — shared inventory improves coordination, but some items/spending can be personal; membership change increases access-control expectations.
- T4: **Reminders vs notification fatigue** — alerts reduce shortages/expiration waste but increase disabling risk; transition-based alerts and deduplication are required.
- T5: **Simplicity vs "smartness"** — fixed thresholds and explainable triggers are robust; predictive forecasting is fragile under missing/noisy data and variable consumption.
- T6: **Automation reliability vs user-device dependence** — scheduled sweeps improve reliability but require observability (run summaries/logs, idempotency, error handling).

3.1.5.10. Traceability hooks (how this supports later sections)

To support traceability (class topic 4.4), later requirements should link to:

- Needs IDs (N1–N5)
- Domain concepts (from 3.1.1)
- Scenarios (3.2.1) and walkthrough findings (3.2.2)
- Verification artifacts (test cases in 3.2.3)
- Optional: related issue IDs once user stories/epics are stabilized

This section establishes the vocabulary used for those links.

3.1.5.11. Requirement quality checks (initial)

Completeness (under-specified) * Define a concrete near-expiry window (N days) and any grace period policy. * Decide initial scope for batch handling (multiple expirations in bulk packs). * Specify shared-household coordination semantics: what "intent/responsibility" means (claim vs assign vs unassigned) and how conflicts are handled. * Define policy for missing/unreadable expiration dates (no alerts vs prompt vs "unknown expiry" status).

Consistency (avoid ambiguous terms) * Avoid "real-time" as a hard requirement unless measurable criteria are provided; prefer "shared visibility" + later quality requirements. * Keep terminology consistent: item type vs item instance/package; inventory (reality) vs tracked record (representation); backup as practice vs state. * Ensure low-stock/out-of-stock/near-expiry/expired

terms map consistently to actionable states and transition rules.

Feasibility (risks/unknowns) * Deduplication rules are essential to prevent notification fatigue and repeated grocery list additions. * Automation sweeps must be testable deterministically with boundary cases (threshold equality, expiration boundaries, missing fields). * Offline/temporary connectivity gaps create reconciliation complexity; initial scope should define acceptable behavior and verification strategy.

3.2. Validation and Verification

Validation ensures we are building the right thing (meets real needs). Verification ensures we built it correctly (meets requirements/specs).

3.2.1. Scenarios

Scenarios describe realistic end-to-end situations in the domain and how stakeholders expect things to work.

NOTE	Expiration-related scenarios and tests refer to the expiration state vocabulary defined in `alert-system-states.adoc` (Active, Expiring, Expired, Renewed, Deleted) and its example alert intervals (e.g., 7/3/1 days). Delivery infrastructure details (summarized in `tech-expiration-alert-system.adoc`) are treated as implementation considerations and are not assumed in scenarios unless explicitly stated.

3.2.1.1. Scenario S1 — Low-Stock Trigger After Consumption

- Persona/actor: Julian, the health-conscious student (primary); household member.
- Context: Julian keeps a personal shelf of meal-prep items separate from roommates. He depends on specific brands that require advance ordering and wants to avoid unexpected shortages during the week.
- Trigger: Julian records that he consumed the last units of a pantry item (e.g., gluten-free pasta), causing the tracked quantity to fall below its low-stock threshold.
- Expected state transition: NORMAL → LOW_STOCK (and Restock list/planner entry created)

Steps

1. Julian opens the system after meal prep and selects the item “Gluten-free pasta” from his inventory.
2. Julian records consumption by decreasing the item quantity (e.g., from 2 boxes to 0).
3. The system detects that the updated quantity is below the defined low-stock threshold and flags the item as low-stock.
4. The system adds the item to the restock list/planner with an initial status of “Needs restock”.
5. The system records and presents a low-stock alert to Julian (delivery method TBD), including the item name.

Expected outcome

- The inventory reflects the updated quantity and low-stock status for the item.
- The item appears in the restock list/planner as needing replenishment.
- Julian becomes aware of the low-stock state in time to reorder before running out mid-week.

Related needs / stories

- Needs: N1, N2
- User stories:
  - US-ALT-01 (Receive low-stock warnings)
  - US-INV-01 (Add items to inventory)

3.2.1.2. Scenario S2 — Shared Inventory Prevents Duplicate Purchase

- Persona/actor: Sofia, the budget-conscious roommate (primary); Andrea, the unofficial household manager (secondary); household members.
- Context: Sofia and her roommates share household essentials. Past issues include duplicate purchases due to out-of-date whiteboards and unclear responsibility for shared items.
- Trigger: Sofia is about to purchase a shared essential (e.g., toilet paper) and checks the shared inventory before paying.
- Expected state transition: No stock-state change for “toilet paper” (remains IN_STOCK); shared visibility converges after synchronization

Steps

1. Sofia opens the shared inventory at the supermarket and searches for “toilet paper.”
2. The system displays the current shared quantity and stock status (e.g., “In stock: 24 rolls”).
3. Sofia sees the item is sufficiently stocked and decides not to buy it.
4. Later, Andrea records a purchase for dish soap; the shared inventory updates for household members once the change is recorded and synchronized.

Expected outcome

- Sofia avoids buying a duplicate shared item, staying within budget.
- Household members converge on the same understanding of the shared inventory after changes are recorded.

Related needs / stories

- Needs: N3, N4, N5
- User stories:
  - US-INV-02 (See what belongs to whom)
  - US-SB-01 (Split shared purchase with roommates)
  - US-RPT-01 (See household dashboard with insights)

3.2.2. Walkthroughs

Walkthroughs are a validation activity: they check whether the team's understanding of the domain and scenarios matches stakeholder expectations. They do not verify implementation correctness (that is addressed by testing).

3.2.2.1. Walkthrough plan

- Participants:
  - Development team members (facilitators and note-takers).
  - 2–3 prospective users acting as domain informants (e.g., students managing shared living spaces).
- Materials:
  - Event table and Important Event Sequences from the domain description.
  - Scenario descriptions (e.g., low-stock detection, expiration handling).
  - Draft UI mockups or simple screen sketches (used only to anchor discussion, not evaluate design).
  - Terminology list defining key concepts such as household, inventory, item type/item instance, restock list/planner, and spending tracking.
  - Expiration state model summary (states and intervals) from `alert-system-states.adoc`.
  - Technical delivery considerations for expiration alerts from `tech-expiration-alert-system.adoc` (used only to sanity-check feasibility; not evaluated with stakeholders).
- Method:
  - Scenario-driven walkthrough using a think-aloud approach (participants verbalize what they expect and why at each step).
  - Facilitator presents a situation and asks participants to describe what they expect to happen next.
  - Event sequence is compared live against participant expectations and domain terminology.
  - Clarifications and disagreements are documented as validation findings.
  - No implementation details are discussed. The focus remains on domain correctness and stakeholder mental models.

3.2.2.2. Walkthrough questions (draft)

- What part of this process feels different from how you normally manage items at home?
- Does this sequence of events reflect what actually happens when something runs out or runs low?
- At this point in the scenario, what would you expect the system-to-be to do next?
- Are any terms (e.g., restock list/planner, low-stock) unclear or unnatural?
- Would you ever ignore or bypass this step in real life? Why?
- Is there information you would need that is not represented here?

- Is this the order in which these things would normally occur?
- When would this feature become annoying or unnecessary (e.g., too many prompts)?

These questions aim to validate mental models and domain assumptions, not measure satisfaction.

3.2.2.3. Findings (examples / initial hypotheses)

Finding	Evidence (participant/session)	Impact / action
Low-stock should not trigger immediately after every quantity decrease.	Example hypothesis to validate: participants may not expect an alert unless quantity crosses a chosen threshold (or remains low). Record quotes with participant code and date (e.g., P1, 2026-02-xx).	Clarify low-stock meaning in terminology; specify threshold-based triggering and deduplication rules.
Expiration does not always imply unavailability.	Example hypothesis to validate: participants may treat expiration differently by item type (e.g., canned goods vs medicine). Record evidence with participant code and date.	Separate “expired” from “unavailable” states; require a policy for what happens on expiration (inform vs block vs manual confirmation).
Restock list/planner should allow manual override.	Example hypothesis to validate: participants may not want all low/expired items auto-added. Record evidence with participant code and date.	Specify rules for manual remove/defer and for auto-add gating to reduce clutter.
Shared-household alerts may be excessive.	Example hypothesis to validate: multiple household members receiving redundant alerts may be perceived as noisy. Record evidence with participant code and date.	Add preferences or scoping (who gets alerted); emphasize grouping/deduplication and “transition-only” alerting.

3.2.3. Testing

***Testing is the primary verification mechanism. For Milestone 1, describe** the test strategy and initial test ideas.*

3.2.3.1. Verification strategy (draft)

- Code review:
  - All changes to main must go through a pull request reviewed by at least 1 teammate not authoring the change.
  - Review focuses on: correctness against requirements/events, edge cases (threshold/expiration boundaries), naming consistency with terminology, and avoiding breaking shared inventory behavior.
  - Review checklist includes: “Does this change preserve event sequence expectations?”, “Does it introduce ambiguous terms?”, and “Are tests added/updated?”
- Automated tests:

    - Run on every pull request and on main branch merges.
    - Start with unit tests for domain rules (quantity updates, low-stock detection, expiration state transitions).
    - Add integration tests for persistence and retrieval (create item → inventory reflects it; remove item → updated totals; shared inventory sync).
    - Add verification tests for automation and deduplication rules (scheduled checks do not create duplicate alerts/list additions).
    - Enforce a minimum baseline: tests must be green before merge.
- Manual checks:
    - Short, repeatable “smoke checks” for milestone features after merging to main.
    - Manual checks prioritize stakeholder-visible behavior: adding/removing items updates inventory, alerts appear under correct conditions, shared inventory reflects changes across accounts.
    - Manual checks are documented as a small checklist so results are reproducible.

3.2.3.2. Test types

- Unit tests:
    - Validate isolated business rules and computations:
        - Handling of missing/unreadable expiration dates (expiry treated as unknown; no false-expired alerts) (US-INV-01 / expiration behavior).
        - Threshold crossing logic and custom thresholds per item (US-ALT-01).
        - Ownership state transitions personal <→ shared (US-INV-02).
        - State transition logic for LOW_STOCK / OUT_OF_STOCK / NEAR_EXPIRY / EXPIRED where applicable (US-ALT-01 / expiration behavior).
        - Transaction creation for quantity changes (restock/consume/adjust) with reason + timestamp (transaction tracking).
- Integration tests:
    - Validate storage + retrieval + computed fields:
        - Item creation persists and appears in inventory query (US-INV-01).
        - Editing an item updates dependent fields and is visible without re-adding (US-INV-01).
        - Inventory view includes ownership indicator and details (US-INV-02).
        - Shared inventory consistency across household members after synchronization (US-INV-02 / shared behavior).
        - Scheduled sweep persists new actionable states and creates notification records/events once per transition (automation behavior).
- End-to-end tests:
    - Validate complete user workflows through API/UI boundaries:
        - Add items → inventory updates → edit item → inventory reflects changes (US-INV-01).

        - Reduce quantity below threshold → low-stock indicator appears and stays consistent (US-ALT-01).
        - Run scheduled check → items with low/expired status generate exactly one new alert per transition (automation + deduplication).
- UI tests:
    - Verify critical screens show correct state (not pixel-perfect):
        - Main screen contains “Add Item” entry point (US-INV-01).
        - Inventory list shows ownership indicator (US-INV-02).
        - Low-stock indicator shown when low (US-ALT-01).
        - Dashboard cards (later milestone) reflect computed totals (US-RPT-01).

3.2.3.3. State model coverage (verification support)

The alert/state vocabulary used in verification is aligned with the project design notes:

- Stock-related actionable states: NORMAL/IN_STOCK, LOW_STOCK, OUT_OF_STOCK (as used by threshold logic).
- Expiration-related states: Active, Expiring, Expired, Renewed, Deleted (from `alert-system-states.adoc`), with configurable alert intervals (e.g., 7/3/1 days) and transition-based alerting to avoid repeated notifications.

Implementation and infrastructure options for delivering expiration alerts (scheduler, background jobs, push delivery) are discussed in `tech-expiration-alert-system.adoc`. Verification focuses on state transitions and deduplication behavior regardless of delivery mechanism.

3.2.3.4. Initial test cases (draft)

Test case	What it verifies	Related requirement/story
TC-INV-01 Add Item entry point exists	Main screen exposes an “Add Item” action as a primary control reachable immediately after opening the app.	US-INV-01 (AC: “Add Item button right on the main screen…”)
TC-INV-02 Create item appears immediately	After saving an item, it is returned in the inventory list without requiring a manual refresh, including created/added date.	US-INV-01 (AC: “appears in his inventory immediately”; “shows up with the date he added it”)
TC-INV-03 Expiration date optional + unknown handling	If expiration date is blank or unreadable, the system treats expiration as unknown (no false-expired alerts); if provided, it persists exactly.	US-INV-01 (expiration behavior)

Test case	What it verifies	Related requirement/story
TC-INV-04 Edit item from inventory list updates dependencies	Editing an item directly from the list updates stored name/metadata and any dependent computed fields that rely on it.	US-INV-01 (AC: “can edit the item right from the inventory list… updates anything else that depends…”)
TC-INV-05 Ownership indicator visible for each item	Inventory list returns an ownership marker for every item (personal owner identifier vs “Shared”).	US-INV-02 (AC: “every item shows a small indicator of who it belongs to…”)
TC-INV-06 Item detail shows ownership + purchase info	Item detail view contains ownership, purchase date, and cost fields when available; shared vs personal status is explicit.	US-INV-02 (AC: detail includes “bought it on… cost… marked as personal/shared”)
TC-INV-07 Change item from personal to shared propagates consistently	Changing an item ownership from personal→shared updates the record and becomes visible consistently wherever the item appears after synchronization.	US-INV-02 (AC: “change it from personal to shared… updates it for everyone…”)
TC-ALT-01 Low-stock indicator triggers on threshold crossing	When quantity drops below the item’s threshold, the low-stock state is set and a low indicator is shown on next app open/view.	US-ALT-01 (AC: “drops below a level he set… next time he opens the app… indicator”)
TC-ALT-02 Custom thresholds persist per item	User-defined thresholds override defaults and persist across sessions; threshold logic uses the custom value.	US-ALT-01 (AC: “he can set custom thresholds… system remembers”)
TC-ALT-03 Low-stock notifications are not spammy (deduplication/grouping)	When multiple items are low, the system avoids repeated alerts without state transitions and may group summaries rather than generating separate alerts per item.	US-ALT-01 (AC: “doesn’t overwhelm… might group them into one message”)
TC-AUTO-01 Scheduled sweep sets LOW_STOCK / EXPIRED correctly	A scheduled check evaluates threshold and expiration boundaries correctly (quantity == threshold; expiration date == today).	Automation behavior (scheduled sweep)
TC-AUTO-02 Alert deduplication on unchanged state	Running the scheduled check repeatedly without any item state transitions does not create duplicate alerts or duplicate list additions.	Automation behavior + deduplication

Test case	What it verifies	Related requirement/story
TC-EXP-01 Expiration state transitions (Active → Expiring → Expired)	Given an item instance with an expiration date, the system assigns Active by default, transitions to Expiring when within the configured near-expiry window/intervals, and transitions to Expired on/after the expiration date boundary.	Expiration behavior (state model)
TC-EXP-02 Renewed/Deleted behavior disables prior expiration alerts	When an item's expiration date is updated (renewed) or the item is removed (deleted), prior pending expiration alerts are not re-emitted, and future alerts follow the new state/interval rules (transition-based, deduplicated).	Expiration behavior + deduplication
TC-TXN-01 Quantity change creates a transaction record	Any quantity change (consume/restock/adjust) creates an append-only transaction record with reason + timestamp; current quantity remains consistent with the transaction history.	Transaction tracking (traceability)
TC-SB-01 Shared vs personal spending separation (data-level)	Purchases tagged shared vs personal are stored distinctly so later splits and spending summaries can be computed correctly.	US-SB-01, US-INV-02 (AC: "separates her personal purchases from shared ones")
TC-RPT-01 Dashboard aggregates update after changes (future milestone)	Dashboard counters update after items/purchases change (no stale computed totals shown once changes are recorded and synchronized).	US-RPT-01 (AC: dashboard reflects updates after changes)

3.2.3.5. Definition of Done

- Code meets the acceptance criteria for the targeted user story (explicit AC mapping is documented in the PR).
- Automated tests added/updated for the acceptance criteria affected; CI passes.
- Any domain terminology or rule changes (ownership, low-stock thresholds, expiration handling, transaction rules) are reflected in documentation.
- No unresolved review issues related to correctness, edge cases, or traceability to US-INV/US-ALT/US-SB/US-RPT stories.

3.2.4. Core Inventory Data Integrity Rules (Verification Support)

This document lists data integrity rules and invariants that will be verified through tests and reviews. The purpose is to ensure that information recorded about household inventory remains consistent, accurate, and meaningful as it changes over time.

In this document, “validation rules” are constraints applied to fields and relationships to prevent inconsistent or misleading records. These rules are written at the system/information level (not tied to any specific database product), so they remain applicable even as implementation details evolve.

3.2.4.1. Terminology alignment (mapping to domain concepts)

The following names are used in this file to match the project’s domain concept analysis:

- **Household** (previously “Home”): a living environment (dorm, apartment, shared house).
- **Household member** (previously “Owner”): a person associated with items (may or may not be a system user).
- **Location** (previously “Room”): a storage place within a household (kitchen, pantry, bathroom, car trunk, etc.).
- **Item type** (previously “Product”): a definition of an item independent of specific packages (e.g., “Milk”, “Toilet paper”).
- **Item instance** (previously “Item/Stock entry”): a specific tracked package/instance of an item type, associated with a household member and location.

3.2.4.2. General integrity principles

- Required fields must be present and valid before saving.
- Text fields must be trimmed (no leading/trailing whitespace).
- Empty strings are not allowed for required text fields.
- Dates must follow logical ordering constraints.
- Numeric fields must follow valid ranges (e.g., quantities cannot be negative).
- If an attribute is optional, NULL is allowed (instead of using an empty string).
- Relationships must be referentially consistent (references point to existing records).

3.2.4.3. System User integrity rules

Represents a system user who can participate in one or more households and manage inventory information.

email

- Required: Yes
- Rule: Must be a non-empty string after trimming.
- Constraint: Must contain "@" and a domain portion (example: user@gmail.com).
- Length: 3 to 254 characters.

- Notes: Must be stored in lowercase for consistency.

name

- Required: Yes
- Rule: Must be a non-empty string after trimming.
- Length: 1 to 50 characters.

created_at

- Required: Yes
- Rule: Must be a valid timestamp.
- Default: current timestamp.

NOTE	Implementation-only fields such as auto-generated identifiers (e.g., integer IDs) are intentionally omitted here, because they do not affect domain correctness. Identifier format and generation belong in the implementation section.

3.2.4.4. Household member integrity rules

Represents a household member associated with items. A household member may or may not have a system user account (e.g., a roommate without an account).

name

- Required: Yes
- Rule: Must be a non-empty string after trimming.
- Length: 1 to 50 characters.

created_at

- Required: Yes
- Rule: Must be a valid timestamp.
- Default: current timestamp.

3.2.4.5. Household integrity rules

Represents a household environment that contains locations and tracked inventory.

name

- Required: Yes
- Rule: Must be a non-empty string after trimming.
- Length: 1 to 100 characters.

created_at

- Required: Yes
- Rule: Must be a valid timestamp.
- Default: current timestamp.

3.2.4.6. Location integrity rules

Represents a storage place within a household (e.g., kitchen, pantry, bathroom).

name

- Required: Yes
- Rule: Must be a non-empty string after trimming.
- Length: 1 to 50 characters.

household_reference

- Required: Yes
- Rule: Location must be associated with exactly one household.
- Constraint: The referenced household must exist.

created_at

- Required: Yes
- Rule: Must be a valid timestamp.
- Default: current timestamp.

3.2.4.7. Item type integrity rules

Represents an item definition independent of specific packages.

name

- Required: Yes
- Rule: Must be a non-empty string after trimming.
- Length: 1 to 100 characters.

brand

- Required: No
- Rule: If provided, must be a non-empty trimmed string.
- Length: 1 to 80 characters.

size

- Required: No

- Rule: If provided, must be a non-empty trimmed string.
- Length: 1 to 40 characters.

variant

- Required: No
- Rule: If provided, must be a non-empty trimmed string.
- Length: 1 to 60 characters.

barcode

- Required: No
- Rule: If provided, must be a trimmed string with only digits.
- Length: 8 to 14 characters (supports common UPC/EAN lengths).
- Notes: Store as text to preserve leading zeros.

created_at

- Required: Yes
- Rule: Must be a valid timestamp.
- Default: current timestamp.

3.2.4.8. Item instance integrity rules

Represents a specific tracked package/instance of an item type, associated with a household member and stored in a location.

item_type_reference

- Required: Yes
- Rule: Item instance must reference exactly one item type.
- Constraint: The referenced item type must exist.

household_member_reference

- Required: Yes
- Rule: Item instance must be associated with a household member (owner/personal association).
- Constraint: The referenced household member must exist.

location_reference

- Required: Yes
- Rule: Item instance must be stored in exactly one location.
- Constraint: The referenced location must exist.

quantity

- Required: Yes
- Rule: Must be an integer.
- Range: quantity >= 0.
- Notes: quantity == 0 is allowed to represent OUT_OF_STOCK rather than forcing deletion.

purchase_date

- Required: No
- Rule: If provided, must be a valid date.
- Constraint: purchase_date should not be in the far future (recommended).
- Recommended check: purchase_date ⇐ today.

expiration_date

- Required: No
- Rule: If provided, must be a valid date.
- Date logic: If both purchase_date and expiration_date are provided, expiration_date must be >= purchase_date.
- Notes: If expiration_date is missing or unreadable, expiration status should be treated as unknown (no false-expired alerts).

notes

- Required: No
- Rule: If provided, must be a trimmed string.
- Length: 1 to 300 characters.

created_at

- Required: Yes
- Rule: Must be a valid timestamp.
- Default: current timestamp.

3.2.4.9. Household membership integrity rules (user–household relationship)

Represents the association between system users and households they can access/manage.

user_reference

- Required: Yes
- Rule: Must reference an existing system user.

household_reference

- Required: Yes
- Rule: Must reference an existing household.

role

- Required: Yes
- Rule: Must be a non-empty trimmed string.
- Allowed values (recommended): "member", "admin".
- Default: "member".

NOTE	Membership changes (e.g., a roommate moving out) should be represented by updating this relationship so access can be removed. The policy for history retention (what happens to past records) is a requirements decision and should be documented in the requirements section.

4. Specific Topics

4.1. Stakeholder Identification

This section identifies stakeholders relevant to the domain and summarizes their goals, influence, and concerns.

4.1.1. Initial Stakeholders

Initial set of stakeholders identified at the start of the project.

Stakeholder	Goals / needs	Influence	Concerns / notes
Student living alone	Keep track of personal groceries, avoid food waste, and manage a tight budget efficiently.	High	Forgetting what items are currently in stock, letting perishable food expire before consumption.
Student living with roommates	Coordinate shared groceries, maintain visibility of common household items, and avoid buying duplicates.	High	Lack of communication leading to excess inventory and wasted space in shared areas.
Roommate who shares purchases	Track who bought what and when, ensuring equitable splitting of common goods like toilet paper, cleaning supplies.	High	Paying more than their fair share, or unexpectedly running out of essential shared items.
Budget-conscious student	Track consumption rates to predict expenses and minimize spending on unnecessary or duplicate items.	Medium	Overspending due to poor inventory visibility and unclear tracking of shared household purchases.
Student with dietary/medical needs	Strictly monitor specific food items or medications, ensuring continuous availability of safe and necessary options.	Medium	Running out of essential specialized items; potential cross-contamination confusion in shared household tracking.
Frequent household supply buyers	Fast data entry and easy management of bulk items to keep the inventory accurate without excessive effort.	Medium	Inventory entry taking too much time, making the application a burden rather than a helpful tool.

Stakeholder	Goals / needs	Influence	Concerns / notes
Parents/Guardians	Ensure the student has enough food and supplies; potentially view shared shopping lists to help fund purchases.	Low	Money being wasted on expired food or duplicate purchases due to poor planning.
Project managers / Course staff	Evaluate project execution, verify domain requirements are met, and assess the software engineering process.	High	The project failing to meet academic standards, syllabus requirements, or adequately solving the proposed domain problem.

4.1.1.1. Stakeholder categories

- **Primary stakeholders:** Student living alone, Student living with roommates, Roommate who shares purchases.
- **Secondary stakeholders:** Budget-conscious student, Student with dietary/medical needs, Frequent household supply buyers, Parents/Guardians.
- **External stakeholders:** Project managers / Course staff.

4.1.1.2. Open questions

- How will users handle the transition of a "personal" item becoming a "shared" item if roommate agreements change?
- What is the threshold for a household supply to be considered "low stock" across different users' habits?

4.1.2. Additional Stakeholders

Stakeholders discovered later as the team learns more about the domain.

NOTE

Lecture topic task summary (Non-Obvious Stakeholders): As part of a lecture topic task on stakeholder identification, we expanded stakeholder analysis beyond the direct users (students living alone or with roommates) to include non-obvious stakeholders whose perspectives affect the domain and requirements. The task identified: (1) **Former roommates** (orphaned items, ownership transfer, move-out access removal), (2) **Future roommates** (pre-move-in planning and duplicate-avoidance via shareable inventory snapshots), (3) **Guests/visitors** (untracked consumption causing “shrinkage” and the need for reconciliation mechanisms), (4) **Parents/family contributors** (bulk/care-package contributions, visibility into needs without full access), (5) **Landlords/property managers** (landlord-owned vs tenant-owned items, restricted items, appliance maintenance), and (6) **Roommates assigned to live together who are strangers** (pre-household formation coordination and fair contribution tracking).

Across these stakeholders, three recurring phenomena emerged: **item ownership**

(including transfer/relinquish/claim workflows), **inventory accuracy under external consumption**, and **notification timing** (alerts useful for contributors and coordination, but risky for noise). These findings motivate a flexible ownership model, multi-phase access (planning vs active household vs post-occupancy), limited external contributor views, and explicit handling of discrepancies/shrinkage and membership changes.

Stakeholder	Goals / needs	Influence	Concerns / notes
Landlords / Property Managers	Ensure that tenant households are kept in good order and that shared spaces are not overloaded or misused.	Low	Unclear lines between personal and shared household property; no formal mechanism for landlord oversight.
Grocery / Retail Stores	Attract repeat purchases by students and household groups; provide accurate product data (barcodes, pricing) that inventory systems may rely on.	Low	Incorrect or inconsistent product metadata in retail databases causing failed barcode lookups.
Mobile Platform Reviewers (App Store / Google Play)	Ensure the mobile application complies with platform guidelines regarding user privacy, data handling, and interface standards before public distribution.	High	App rejection due to improper handling of user data or failure to meet minimum functionality requirements for listing.

4.2. Personas

Personas represent key user/stakeholder types and are the single source of truth for all requirements, user stories, and features. Each persona includes goals, frustrations, behaviors, constraints, and a short scenario. The persona-to-needs trace table at the end of this file links each persona to specific needs (N1, N2, etc.), which are referenced throughout requirements for justification and traceability.

4.2.1. Persona 1: Mateo, The Busy Solo Student

- **Role:** Student living alone
- **Background:** Mateo is a full-time engineering student who also works a part-time job. He lives by himself in a small apartment near campus and has very little free time or mental energy for household management.
- **Goals:**
  - Minimize time spent grocery shopping.
  - Avoid throwing away expired perishable food.

    - Keep meal prep quick, simple, and predictable.
- **Frustrations:**
    - Frequently forgets what ingredients he already has in the fridge.
    - Buys fresh produce or meat, but it goes bad before he finds the time to cook it.
    - Experiences morning stress when he realizes he is completely out of coffee or milk.
- **Typical behaviors:**
    - Shops sporadically, usually only when the fridge is completely empty.
    - Relies entirely on his memory for his home inventory.
    - Orders expensive takeout when overwhelmed by a disorganized kitchen.
- **Constraints**:
    - Severe time constraints due to a heavy class and work schedule.
    - Limited mental bandwidth for tracking chores; relies heavily on his smartphone for organization.
- **Scenario:** Mateo comes home late after a grueling day of classes. He remembers buying chicken breasts earlier in the week and plans to quickly pan-fry them for dinner. When he opens the fridge, he realizes he actually bought them almost a week and a half ago, and they have spoiled. Frustrated, he orders an expensive pizza delivery, breaking his weekly food budget, and later realizes he also has three open bottles of ketchup in the pantry. Success for Mateo looks like receiving an alert/notification (delivery TBD / in-app baseline) the day before his chicken expires, and being able to quickly check an app at the supermarket to confirm he doesn't need to buy a fourth bottle of ketchup.

4.2.2. Persona 2: Sofia, The Budget-Conscious Roommate

- **Role:** Student living with roommates and Budget-conscious student
- **Background:** Sofia lives in a shared apartment with two other students. She pays for her own living expenses using a mix of financial aid and a strict monthly budget.
- **Goals:**
    - Keep grocery and household supply costs as low as possible.
    - Ensure communal items like toilet paper and dish soap are split fairly.
    - Avoid buying any duplicates of items the apartment already has.
- **Frustrations:**
    - Roommates using her specific, more expensive dietary items without asking.
    - Running out of money before the end of the month due to poor collective household planning.
    - Wasting limited apartment storage space on duplicate bulk items.
- **Behaviors:**
    - Shops with a strict list and a calculator app open.
    - Tries to use a physical whiteboard on the fridge to track shared apartment needs, but her

roommates forget to update it.

- **Constraints:**
  - Extremely limited financial budget.
  - Shared physical storage space that fills up quickly.
  - Reliant on uncooperative roommates to manually update the physical whiteboard.
- **Scenario:** Sofia goes to the supermarket with a strict $50 budget. She checks the apartment's shared whiteboard and sees they need toilet paper and dish soap. She buys them, using up a significant chunk of her cash. When she returns, she finds one of her roommates had already bought bulk toilet paper earlier that afternoon but forgot to erase it from the whiteboard. Now the apartment has no storage space left, and Sofia is short on cash for her own meals. Success for Sofia looks like a synchronized mobile app where changes are recorded and synchronized so everyone sees updated shared inventory and purchases, preventing her from spending her limited money on duplicate items.

4.2.3. Persona 3: Julian, The Health-Conscious Student

- **Role:** Student with dietary or medical needs
- **Background:** Julian is a student athlete who follows a strict, high-protein, gluten-free diet. He lives with roommates but keeps his food entirely separate due to his nutritional requirements and fear of cross-contamination.
- **Goals:**
  - Maintain a consistent, uninterrupted supply of specific, specialized dietary items.
  - Accurately track consumption rates of expensive supplements to budget for reorders.
- **Frustrations:**
  - Running out of core meal-prep ingredients mid-week.
  - Disorganized pantry spaces making it hard to quickly find what he needs.
  - Specialty items expiring because he bought too much in bulk to save money online.
- **Behaviors:**
  - Buys non-perishables in bulk online to save money.
  - Does heavy meal prep every Sunday afternoon.
  - Tracks his macronutrients religiously but struggles to translate that into a grocery inventory.
- **Constraints:**
  - Strict dietary policies that don't allow for substituting ingredients.
  - Limited local availability of his required brands.
- **Scenario:** Julian does his weekly meal prep on Sunday. Halfway through cooking, he goes to his pantry shelf to grab his special gluten-free pasta, only to realize he used the last box on Wednesday and forgot to order more. Because the local grocery store doesn't carry his brand, his entire meal plan is ruined. Success for Julian looks like threshold-based, adjustable low-stock alerts that remind him to reorder specialty items before he runs out.

4.2.4. Persona 4: Andrea, The Unofficial Household Manager

- **Role:** Frequent household supply buyer and Roommate who shares purchases
- **Background:** Andrea lives in a house with three other students. Because she is the most organized and has a car, she naturally fell into the role of the "unofficial house manager," doing the bulk of the household shopping at wholesale clubs.
- **Goals:**
  - Keep the house perfectly stocked with shared essentials without doing all the mental labor.
  - Get reimbursed quickly and accurately by roommates for shared purchases.
- **Frustrations:**
  - Constantly having to text the roommate group chat to ask what they are out of, getting no replies.
  - Getting stuck paying the bill when shared items are consumed but nobody admits to finishing them.
- **Behaviors:**
  - Uses a chaotic mix of WhatsApp messages, scattered receipts, and Apple Notes to track house needs.
  - Frequently sends money requests to roommates after a shopping trip.
- **Constraints:**
  - Coordinates across three different and uncooperative roommate schedules.
  - Needs accurate, unarguable proof of consumption for financial reimbursements.
- **Scenario:** Andrea plans a trip to a wholesale club. She texts her roommates asking what they need, but only one replies. She guesses on the rest and buys shared snacks and cleaning supplies. The shared snacks disappear rapidly, and when she asks for everyone to split the cost, arguments break out over who actually ate them. Meanwhile, the house runs out of trash bags because Andrea assumed they had some hidden under the sink. Success for Andrea looks like a transparent, shared digital ledger where roommates can log what they finish and request items, making reimbursements unarguable and keeping the house perfectly stocked.

4.2.5. Persona-to-needs trace

Persona	Key needs
Mateo	Know what food is currently in stock; receive expiration alerts before items spoil; avoid running out of essential daily items
Sofia	Prevent duplicate purchases in shared households; split shared costs fairly among roommates; stay within a strict monthly budget
Julian	Maintain an uninterrupted supply of specialty dietary items; predict restocking needs using adjustable threshold alerts; keep personal items separate from shared inventory

Persona	Key needs
Andrea	Coordinate shared household stocking without carrying all the mental load; obtain accurate, verifiable records for roommate reimbursement; know what is needed before a bulk shopping trip

4.3. Traceability

This section describes how traceability is established in the Digital Home Inventory project. Traceability means our deliverables are linked so we can answer questions about where requirements come from, which personas they serve, and how they connect to domain concepts, scenarios, and verification artifacts.

4.3.1. Overview

Traceability helps us understand relationships between different parts of our documentation. When something changes, we can see what else might be affected. For example, if a requirement is modified, we can identify which personas and needs it supports, which user stories are impacted, and which scenarios/tests should be revisited.

Our approach focuses on creating clear links between artifacts without overcomplicating the documentation (Milestone 1 focuses on progress, not completeness).

4.3.2. Traceability approach (labels and links)

We maintain traceability through a simple labeling system:

4.3.2.1. Persona goals

Each persona's goals are labeled with a short code. (Goal descriptions are kept short to make them easy to reference.)

- **Mateo (M)**
- **MG1**: Know what I have without extra effort
- **MG2**: Avoid wasting expired food
- **MG3**: Avoid running out of daily essentials
- **Sofia (S)**
- **SG1**: Stay within a strict budget
- **SG2**: Split shared costs fairly
- **SG3**: Avoid duplicate purchases
- **Julian (J)**
- **JG1**: Avoid running out of specialty items
- **JG2**: Avoid waste from bulk purchases

- **JG3**: Keep personal items separate
- **Andrea (A)**
- **AG1**: Coordinate shared stocking without mental overload
- **AG2**: Get reimbursed accurately for shared purchases
- **AG3**: Know what is needed before bulk shopping

4.3.2.2. Needs requirements

Each requirement in Section 2.2 is tagged with which Needs IDs (N1–N5) it supports, and (when helpful) which persona goals it supports.

Example

- **IR-01 (Item Initialization)** → Needs: N1, N2 | Goals: MG1, JG1, AG3
- **IR-04 (Low-stock thresholds)** → Needs: N2 | Goals: MG3, JG1, AG1
- **IR-07 (Cost splitting)** → Needs: N5 | Goals: SG2, AG2

4.3.2.3. User stories requirements

Each user story in Section 2.2.1 lists which interface requirements it implements.

Example

- **US-INV-01** → Implements: IR-01, IR-02, IR-03, IR-05
- **US-ALT-01** → Implements: IR-04
- **US-SB-01** → Implements: IR-06, IR-07
- **US-RPT-01** → Implements: IR-08

4.3.2.4. Domain concepts requirements

Domain concepts from Section 2.1 (Terminology/Entities/Events/Functions) are linked to the requirements that depend on them.

Example

- **Item type / Item instance** → Appears in: IR-01, IR-02, IR-05
- **Household / Household member** → Appears in: IR-03, IR-06, IR-07
- **Low-stock threshold** → Appears in: IR-04
- **Expiration date** → Appears in: IR-05
- **Spending entry / Budget (optional scope)** → Appears in: IR-06, IR-07, IR-08

4.3.2.5. Scenarios/tests requirements (Milestone 1)

For Milestone 1, traceability is extended to the main validation and verification artifacts:

- Scenario IDs: **S1..Sx** (Section 3.2.1)
- Test case IDs: **TC-*** (Section 3.2.3)

- Walkthrough findings: **WF-*** (Section 3.2.2) once real sessions are performed

Example

- **S1 (Low-stock after consumption)** → IR-02, IR-04 → TC-ALT-01, TC-AUTO-01
- **S2 (Avoid duplicate purchase in shared household)** → IR-03, IR-08 → TC-INV-05, TC-RPT-01

4.3.3. Traceability matrix (Milestone 1 snapshot)

Goal	Need(s)	Requirement(s)	Story / Scenario	Domain concept(s)
MG1	N1, N2	IR-01, IR-02	US-INV-01 / S1	Item type, Item instance, Tracked inventory record
MG3	N2	IR-04	US-ALT-01 / S1	Low-stock threshold, Stock state
SG2	N5	IR-06, IR-07	US-SB-01	Spending entry, Shared vs personal
JG1	N1, N2	IR-01, IR-04, IR-05	US-INV-01, US-ALT-01	Item instance, Expiration date, Threshold
AG2	N5	IR-06, IR-07	US-SB-01	Spending entry, Attribution
AG3	N3	IR-08	US-RPT-01 / S2	Dashboard metrics, Shared visibility

4.3.4. Questions traceability helps answer (kept concise)

- Which personas and needs are affected if a requirement changes?
- Which scenarios/tests must be revisited to verify the changed requirement?
- Which domain concepts are involved in a given requirement (to avoid terminology drift)?

4.3.5. Current traceability status (Milestone 1)

We have established traceability for:

- Persona goals: all personas have labeled goals (MG*, SG*, JG*, AG*).
- Needs: core needs N1–N5 are referenced by the interface requirements.
- Requirements: IR-01 through IR-08 are tagged to needs/goals and mapped to stories.
- Validation/verification: key scenarios and initial test cases are linked to the most central IRs.

NOTE	Deferred items for later milestones: * Additional edge-case interface requirements captured under “Deferred edge cases (Milestone 2+)” will be traced once they are promoted into explicit IR IDs. * Domain requirements (DR-*) will be added to the traceability matrix once the DR list is finalized and terminology is normalized

across all requirements.

4.4. Domain Requirements Acquisition

This section documents the methodology, planning, and resources used by the team to gather and validate domain knowledge. It details how we acquired realistic insights into how students manage household inventories, coordinate with roommates, and track expenses.

4.4.1. Requirements Acquisition Plan

Plan for how we will gather and validate requirements from stakeholders and domain evidence. For Milestone 1, this plan is intentionally lightweight and focuses on collecting concrete examples that can be traced to needs, scenarios, and requirements.

4.4.1.1. Goals (Milestone 1)

- Capture real ("messy") household practices instead of idealized descriptions.
- Understand shared-household coordination (visibility, duplicates, ownership/personal vs shared, and cost splitting).
- Identify the main friction points that cause tracking methods to be abandoned.
- Clarify key domain edge cases (bulk purchases, multiple locations, missing/unclear expiration dates).

4.4.1.2. Methods (Milestone 1)

- **Interviews (4–6):** Semi-structured interviews with solo students and students in shared households (including at least one student athlete). Focus on recent, specific incidents (waste, duplicates, shortages, roommate friction).
- **Contextual walkthroughs:** Ask participants to show their current artifacts (fridge notes/whiteboards/messages) and explain their shorthand and terminology.
- **Light competitive scan:** Review a small set of existing inventory/list apps to identify recurring friction patterns (especially entry effort and notification fatigue).

NOTE Surveys and broader quantitative validation are considered useful but may be deferred if time is limited; Milestone 1 prioritizes rich qualitative evidence suitable for domain description and initial requirements.

4.4.1.3. Schedule (Milestone 1)

Timeframe	Activities
Late Jan. – Early Feb.	Conduct initial interviews and contextual walkthroughs; expand the rough sketch with concrete quotes/examples; begin light competitive scan.
Mid-Feb.	Synthesize findings into personas/scenarios; refine terminology and concept analysis; record open questions and clashes.

Timeframe	Activities
Late Feb.	Validate assumptions with follow-up questions (short check-ins); finalize initial user stories and core interface requirements; mark deferred edge cases explicitly.

4.4.1.4. Risks and mitigations (Milestone 1)

- **Risk:** Participants are too busy for long interviews.
  - **Mitigation:** Keep sessions short (15–25 minutes) and allow asynchronous voice-note follow-ups.
- **Risk:** Participants describe ideal behavior rather than actual behavior.
  - **Mitigation:** Ask for recent concrete examples ("last time you threw out food", "last duplicate purchase", "show your last note/chat").
- **Risk:** Manual entry tolerance is near zero, making tracking unsustainable.
  - **Mitigation:** Explicitly measure "effort tolerance" (seconds/taps) and capture what minimum information participants are willing to record.

4.4.2. Resources

Evidence sources and supporting materials used to understand the domain and justify requirements. For Milestone 1, we list only the most important sources; extended notes are linked.

4.4.2.1. External sources (selected)

- Household food waste research: supports the need for better visibility and expiration awareness in student households.
- Personal Information Management (PIM) research: supports the risk that high-effort or overly frequent updates lead to abandonment and notification fatigue.
- Household finance / budgeting research (general): supports the importance of spending awareness for budget-constrained students.

Extended literature notes: `docs/lecture-topic-tasks/domain-acquisition-ltt.adoc`

4.4.2.2. Internal artifacts (project)

- Domain rough sketch and narrative: `docs/2-descriptive/domain-description/rough-sketch/` and `docs/2-descriptive/domain-description/narrative/`
- Terminology table: `docs/2-descriptive/domain-description/terminology/terminology.adoc`
- Domain concept analysis: Section 3.1 (including invariants and term clashes)
- Validation and verification artifacts: Section 3.2 (scenarios, walkthrough plan, testing)

4.4.2.3. Open questions (Milestone 1)

- Interview and walkthrough findings should be summarized and linked here once sessions are completed (participant codes + dates + key quotes).

- Competitive scan results should be summarized (top friction points and why) once completed.

4.5. Inconsistency Management

An exploration of our domain terminology, database schema, events, actions and behaviors, shared inventory definition, and duplicate item criteria led to the discovery of two inconsistencies. * **Event naming clash — "An Inventory has been generated":** This event, triggered by the addition of an item to the inventory for the first time, was described as changing the inventory so that it now reflects the presence of a new item. However, the term *Inventory* is defined in our terminology as "the list of available products in a home," while *Item* (a synonym for *Stock*) is defined as "an instance of a product." The event description created a designation clash with the existing glossary: the name alluded to the creation of an inventory, but the described change was a modification of an already-existing inventory. To resolve this clash, the event was split into two: **The "What changed" description of "An Inventory has been generated" was updated to: "The inventory is created and reflects the presence of the new item." This separates the creation of the inventory from the modification of an existing inventory, and clarifies that the inventory is created as a direct result of adding an item for the first time.** A new event, **"A new Product has been generated"**, was introduced for cases where an item is added to the inventory with no relation to an existing product. This event records the change to the inventory reflecting the new product description as well as the item count and description.

- **Ownership multiplicity mismatch — Group-Owned Inventory model:** The Group-Owned Inventory model in the Shared Inventory Definition document describes how all items can be mapped as belonging to the household, with multiple members able to add, edit, and delete them. It further describes that inventories have multiple stock entries, and that stock entries may optionally carry linked owner tags. This implies that items can have multiple owners. However, the ER and table diagrams for database item representation did not reflect this: the model was only satisfiable when items had exactly one owner.
    - To resolve this weak conflict, the ER and table diagrams for database item representation were updated. The relationship between items and owners was changed to a many-to-many relationship with total participation on both sides. A new table, **Owns**, was added to represent this relationship. This change allows an item to be owned by multiple members while preserving the ability of a member to own multiple items.

Generative AI Disclosure

In our Digital Home Inventory project, we utilized generative AI tools to enhance our workflow while maintaining ownership of the core ideas and decisions. The tools we used were Claude 4.5 (accessed via a free account), ChatGPT (free tier), and DeepSeek AI. These were primarily employed during the brainstorming and structuring phases, where they helped generate initial ideas for project features, such as inventory tracking, notifications for low stock or expirations, budget management, roommate sharing to avoid duplicates, and report generation. For documentation, we used AI to check grammar, suggest edits, and refine the structure to ensure it correlated correctly with rubric criteria, ensuring the documentation met the assignment criteria. The core work originated from our team, starting with original components like our feature analysis, research on requirements (e.g., Supabase schema for items with privacy fields, Flutter UI for scanning), and

project scope discussions. AI was used to enhance these, providing suggestions on implementation steps or alternative structures, but we always edited and revised the output to align with our vision. For instance, when AI suggested feature requirements like API integrations for expiration dates, we reviewed them against the A3 document and team discussions, rejecting or modifying those that didn't fit our student-focused goals. To verify AI-generated content, we fact-checked everything with the team, cross-referencing course materials, rubrics, and real-world examples from similar apps. If inconsistencies arose, such as AI recommendations that conflicted with our privacy feature for personal items, we resolved them through brainstorming sessions, deciding collectively whether to incorporate, edit, or discard. One example: AI proposed a complex budgeting algorithm, but it didn't align with our simple student needs, so we simplified it after team review. From using AI, we learned it streamlined research and idea generation, freeing up time for deeper reflection on what the app truly needed for student users. It didn't reduce engagement, instead it forced us to critically evaluate outputs, enhancing our learning and analytical skills. Overall, AI acted as a supportive tool, not a replacement, helping us iterate faster while keeping the project authentic to our group's efforts.